

3D GameStudio



Tutorial

#3D-Game-Profi in 5 Tagen#

© 1995/Jun 1998 J.C.Lotter / Conitec GmbH

 **CONITEC DATENSYSYSTEME**
64807 Dieburg • Dieselstr. 11c • fon 06071-92520 • fax 06071-925233

Handbuch und Software unterliegen dem Urheberrechtsschutz. Kein Teil dieses Handbuchs darf ohne ausdrückliches schriftliches Einverständnis von conitec reproduziert, vervielfältigt, gespeichert oder übersetzt werden. Für die Richtigkeit der hier angegebenen Daten und Informationen übernehmen wir keine Gewähr; Änderungen, die dem Fortschritt dienen, behalten wir uns vor.

Inhalt

Montag: Konstruktion eines Minimal-Levels 1

Dienstag: Die Player-Bewegung 10

Mittwoch: Portale, Lifte, Wasserfälle 29

Donnerstag: Künstliche Intelligenz 42

Freitag: Panels, Menüs, Overlays 64

Index 89

In diesem 5-Tage-Programm geht es um die WDL-Praxis: Wie konstruiere ich ein professionelles 3D-Game? Der Einfachheit halber wollen wir in diesem Tutorial von folgendem Kenntnisstand ausgehen: Sie haben das Referenz-Handbuch komplett durchgelesen, aber nichts davon verstanden. Zwar können wir in den 5 Lektionen nicht alle der vielfältigen Features des 3D GameStudio behandeln, doch für einen Einstieg, von dem aus Sie weiterarbeiten können, reicht's allemal.

Als Voraussetzung sollten Sie WED.EXE (bzw. WEDS.EXE oder WEDC.EXE) mit sämtlichen Hilfsdateien in einem Verzeichnis Ihrer Harddisk haben, welches im DOS-Pfad liegt. Führen Sie dazu das SETUP-Programm auf der Programmdiskette aus; es wird das Verzeichnis C:\Programme\GStudio und die Unterverzeichnisse SKAPH und DEMO angelegt. Sie werden zuerst mit den Dateien des DEMO-Verzeichnisses arbeiten.

Montag: Konstruktion eines Minimal-Levels

Am ersten Tag lernen Sie, wie Sie Schritt für Schritt einen Level mit GameStudio aufbauen. Der Level soll der Einfachheit halber nur aus ein paar Räumen bestehen.

Zum schnellen Starten und Testen Ihres Levels arbeiten Sie am besten unter DOS. Als Game-Profi gewöhnen Sie sich schnell an das Level-Bearbeiten unter Windows 95 mit dem Texteditor in einem Fenster, dem World Editor WED in einem zweiten und einem Grafikprogramm - z.B. Paintshop Pro - in einem dritten Fenster. Erstellen Sie zuerst mit einem normalen Texteditor - z.B. Wordpad® im Text-Modus - Ihre erste WDL-Datei **MINI.WDL**, welche die Beschreibung für unseren Level enthält.

Was Sie zuerst in der Datei benötigen, ist ein 'Kopf', der die Video-Auflösung und den Namen der WMP- und WDL-Datei festlegt:

```
////////////////////////////////////  
// Minimal-Level  
// created 12.8.1998 (myself)  
////////////////////////////////////  
VIDEO          x320x400;  
MAPFILE        <miniw.WMP>;  
BIND           <mini.WDL>;
```

Alle Zeilen in einer WDL-Datei, die mit doppelten Schrägstrichen beginnen, sind Kommentare. Sie können damit Ihre Definitionen übersichtlicher gestalten. Das **BIND**-Schlüsselwort ist nur ein Hinweis für den Compiler der Profi-Version, daß er die WDL-Datei selbst später zum fertigen Game hinzucompilieren soll. Obwohl Groß- oder Kleinschreibung nicht relevant sind, sollten aus Gründen der Übersichtlichkeit alle Schlüsselworte und vordefinierten Skills GROSS und die selbstdefinierten Namen klein geschrieben werden.

Ebenfalls in den 'Kopf' gehören **INCLUDE**-Anweisungen, mit denen weitere WDL-Dateien hinzugebunden werden sollen:

```
INCLUDE        <move.WDL>;
```

Die Datei **MOVE.WDL** enthält die Aktion **set_walking**, welche die Player-Bewegung initialisiert. Diese WDL-Datei finden Sie auf der Demo-Diskette. Ihr Inhalt soll uns zunächst nicht kümmern, denn saubere Bewegungs-Aktionen sind nicht ganz unkompliziert. Sie können diese Aktion einfach benutzen, ohne sie im Detail zu kennen. Definieren Sie sie als **IF_START**-Aktion, damit sich der Player im Level bewegen kann:

```
IF_START      set_walking;
```

Nun können wir an die eigentliche Definition des Levels gehen. Zunächst geben wir die Farbpalette mitsamt ihren Shading-Bereichen an, die für die Darstellung der Texturen erforderlich ist:

```

PALETTE pal1 {   PALFILE <vrpal.pcx>;
    RANGE 16,16; RANGE 32,16; RANGE 48,16; RANGE 64,16;
    RANGE 80,16; RANGE 96,16; RANGE 112,16; RANGE 128,8;
    RANGE 136,8; RANGE 144,8; RANGE 152,8; RANGE 160,16;
    RANGE 176,16; RANGE 192,16; RANGE 208,16; RANGE 226,16;
    RANGE 240,16;
}

```

Wenn Sie mehrere Paletten definieren, wird die zuletzt definierte bei Spielstart automatisch geladen. Die Palettendatei finden Sie wieder im DEMO-Verzeichnis, ebenso wie die Bitmaps für die Texturen. Vergessen Sie jedoch nicht, daß diese Dateien - im Gegensatz zu denen im SKAPH-Verzeichnis - **nicht lizenzfrei** sind. Sie dürfen sie zum Üben verwenden, aber nicht in eigenen Games einsetzen!

Zur Darstellung eines Raums benötigen Sie mindestens eine Wandtextur und eine quadratische Boden- und Deckentextur:

```

BMAP arc_map, <wandtex.lbm>,320,0,64,128;
BMAP square_map,<bodentex.lbm>,0,0,128,128;
////////////////////////////////////
TEXTURE arc_tex {
    SCALE_XY 10,10;
    BMAPS arc_map;
}
TEXTURE square_ftex {
    SCALE_XY 16,16;
    BMAPS square_map;
}
////////////////////////////////////
WALL arc_wall {
    TEXTURE arc_tex;
    FLAGS PORTCULLIS;
    POSITION 1;
}

```

Die **SCALE_X**, **SCALE_Y**-Werte sagen der Engine, wie die Bitmap 'gezoomt' werden soll - in Einheiten von Pixels pro **Step**, der Größeneinheit Ihrer künstlichen Welt. Auch hier wieder eine Konvention aus Übersichtsgründen: Innerhalb von Definitionen - und ganz allgemein zwischen geschweiften Klammern - rücken wir alle Zeilen um eine Tabulatorspalte ein, um anzudeuten,

daß sie zu einem Oberbegriff gehören. Das **PORTCULLIS**-Flag mit **POSITION 1** haben wir bei der Wand gesetzt, damit gewährleistet ist, daß die Textur direkt an der Decke endet. Eine andere Methode wäre **ALIGN CEILING** im **WED**.

Was nun noch fehlt, sind die Regionen. Obwohl wir zuerst nur einen einzigen Raum darstellen wollen, benötigen wir zwei Regionen, nämlich eine für den Raum selbst und eine für die umgebende 'massive' Levelgrenze:

```
REGION border {  
    FLOOR_HGT 40;  
    CEIL_HGT  40;  
    FLOOR_TEX square_ftex;  
    CEIL_TEX  square_ftex;  
    CLIP_DIST 0;  
}  
REGION dungeon {  
    FLOOR_HGT 0;  
    CEIL_HGT  12;  
    FLOOR_TEX square_ftex;  
    CEIL_TEX  square_ftex;  
}
```

Zu beachten ist, daß bei der **border**-Region die Boden- und Deckenhöhe gleich sein muß, damit unser Level eine auf allen Seiten geschlossene undurchdringliche Grenze bekommt. Da man das 'Innere' der **border**-Region folglich nie sieht, habe wir ihr eine **CLIP_DIST** von 0 gegeben - die Region wird dadurch beim Rendern nicht berücksichtigt. Für diesen kleinen Level wär's egal, aber in komplexen, verschlungenen Leveln erhöht das die Bildaufbaugeschwindigkeit.

Nachdem unsere WDL-Datei **MINI.WDL** fertig ist, erstellen wir nun die Topographie. Hierzu öffnen Sie ein DOS-Fenster, wechseln in das Verzeichnis, in dem Sie **MINI.WDL** abgespeichert haben (das sollte das **DEMO**-Verzeichnis sein), und geben ein:

WED MINI

(Besitzer der Lite-Version schreiben immer **WEDS**, Besitzer der Commercial-Version **WEDC** statt **WED**).

Da WED die im WDL-Kopf angegebene Topographie-Datei **MINIW.WMP** nicht findet, erstellt er automatisch eine neue Datei. Beim Start sehen Sie zunächst nur ein leeres Bildfeld mit einem blauen Punkt in der Mitte: Das ist die Player-Startposition.

Gehen Sie mit **[V]** in den Vertex-Modus und klicken Sie einigemal im Uhrzeigersinn im Abstand von etwa 40 Koordinateneinheiten (Steps) um die Playerposition herum. Jeder Klick positioniert einen **Vertex** in Form eines grünen Kreuzes. Nachdem Sie ein Vier- oder Fünfeck aus Vertices erstellt haben, 'ziehen' Sie nun mit gedrückter linker Maustaste einen Rahmen um alle Vertices. Sobald Sie die Maustaste loslassen, verschwindet der Rahmen, und um jeden Vertex erscheint ein grünes Quadrat. Die Vertices sind nun **markiert**.

Drücken Sie **[Shift-Einfg]**. WED verbindet alle markierten Vertices durch einen geschlossenen Linienzug und schaltet zugleich in den Wall-Modus um. Sie sehen um jede Linie eine grüne Markierung mit einem Querstrich, der - wenn Sie alle Vertices korrekt im Uhrzeigersinn erstellt haben - nach innen zeigen sollte.

Falls sich Linien kreuzen, so haben Sie beim Vertex-Klicken die Reihenfolge nicht eingehalten. Gehen Sie noch einmal in den Vertex-Modus (**[V]**) und verschieben Sie einzelne Vertices mit gedrückter linker Maustaste, bis alles stimmt. Gekreuzte Wände führen im Level zu Darstellungsfehlern! Anschließend schalten Sie mit **[W]** wieder in den Wall-Modus und markieren alle Linien wie gehabt durch Ziehen eines Rahmens.

Sie haben nun ein Vier- oder Mehreck aus grün umrandeten Linien mit 'Nasen', die nach innen zeigen. Die Nase jeder Wand gibt ihre rechte Seite an; sonst hat sie weiter keine Bedeutung. Mit der Taste **[F]** können Sie übrigens die linke und rechte Seite der ausgewählten Wand vertauschen. Tun Sie das, falls irgendwelche Wände in die falsche Richtung zeigen.

Sie müssen jetzt allen Linien einen Wall-Namen zuordnen. Markieren Sie sie wieder mit einem Rahmen, dann klicken Sie rechts. Geben Sie den Wänden den

in der Scroll-Box einzigen vorhandenen Wall-Namen **arc_wall**. Anschließend scannen Sie Ihren Raum per **[O]**-Taste ein, wobei WED zugleich in den Regions-Modus umschaltet.

Wenn Sie jetzt den Cursor in eine geschlossene Region hineinbewegen, leuchtet die Umgrenzung auf (andernfalls ist irgendwo eine Lücke). Da Sie im Moment nur eine einzige umgrenzte Region haben, wird diese immer leuchten. Klicken Sie rechts, und wählen Sie dann eine Region aus der Scroll-Leiste mit Doppelklick oder **[ENTER]**. Mit dem Button **[Alternate/Default]** wechseln Sie zwischen vorher im WED eingegebenen und WDL-Default-Regionshöhen - wählen Sie hier **[Default]** und quittieren anschließend mit **[OK]**. Nehmen Sie **dungeon** für das Innere und **border** für den Außenbereich Ihres Raums.

Kontrollieren Sie anschließend im Wall-Modus durch Berühren der Wände mit dem Mauszeiger, ob alles seine Richtigkeit hat. Die Wand-Informationen werden auf dem Datenpanel unten angezeigt. Insbesondere muss die **border**-Region, die Ihren Level umgibt, unbedingt gleiche Boden- und Deckenhöhe haben, sonst kann die Engine nicht arbeiten! Checken Sie den Level mit dem Menüpunkt **[CHECK WALLS]**. Wenn Sie irgendeine Wand oder Region nicht oder falsch zugeordnet haben, wird der fehlerhafte Bereich markiert.

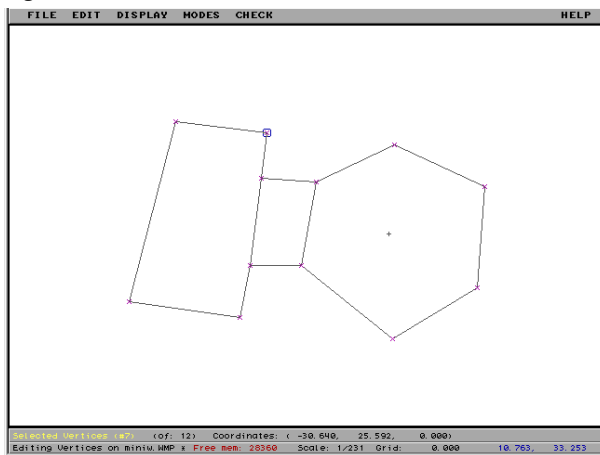
Nun sind Sie mit Ihrem ersten Mini-Level fertig... oder jedenfalls fast. Sie müssen Ihren Level noch mit **[F2]** abspeichern. Starten Sie den Probelauf mit **[Shift-^]**. Wenn alles geklappt hat, werden Sie nun in Ihren Raum 'hineingebeamt'. Solange Sie noch keine eigenen Tasten definiert haben, können Sie mit **[F5]** die Bewegungsunschärfe umschalten (die Wände 'pixeln' dann nicht so stark), mit **[F2]** und **[F3]** Ihre Position speichern und zurückladen, mit **[F6]** einen Screenshot abspeichern, mit **[Shift-^]** zwischen Game und WED hin- und herschalten oder mit **[F10]** das Game beenden. Schauen Sie sich aber vorher in Ihrem selbst-definierten Raum gründlich um! Vermutlich werden Sie als erstes feststellen, daß die Wandtexturen an den Ecken unschön abgeschnitten sind.

Das wollen wir noch schnell korrigieren: Schalten Sie in den WED zurück, aktivieren Sie den Wall-Modus (**[W]**), markieren Sie sämtliche Außenwände Ihres Raumes durch Anklicken mit gedrückter **[Shift]**-Taste oder durch Ziehen eines Rahmens, und wählen Sie dann den Menüpunkt **WALL - ALIGN HOR**.

Sie sehen nun unten im Info-Panel, daß sich am Wert 'x y offs' etwas getan hat. Die Wände haben einen X-Offset bekommen, d.h. ihre Texturen wurden horizontal verschoben. Den Wert dieser Verschiebung hat WED automatisch so berechnet, daß die Texturen nun nahtlos aneinanderpassen. Starten Sie Ihr Level erneut mit **[Shift-A]**, und überzeugen Sie sich davon. Je nachdem, wie lang Ihre Wände sind, kann es prinzipbedingt an der Berührungsstelle der allerersten mit der letzten Wand immer noch eine 'Naht' geben. Dies könnten wir durch Verändern der Länge der letzten Wand oder der horizontalen Textur-Skalierung ausbügeln, wollen es jedoch jetzt für's erste gut sein lassen.

Nun, unser aus einem einzigen Raum bestehende Level sieht denn doch noch etwas mickrig aus. Fügen wir einen weiteren Raum hinzu! Er soll einen niedrigeren Boden und eine höhere Decke haben als der erste.

Bauen Sie den neuen Raum wie gehabt in einem Abstand von etwa 10 Einheiten neben den ersten. Nun müssen wir beide Räume noch durch einen kleinen Gang verbinden. Markieren Sie zwei gegenüberliegende Vertices - je einer vom ersten und einer vom zweiten Raum - und erzeugen Sie eine Verbindungslinie mit **[Einf]**. Vertices können Sie übrigens auch im Wall-Modus erzeugen, indem Sie mit dem Cursor eine Wand berühren und diese mit **[S]** teilen - in der Mitte der Wand wird dann automatisch ein neuer Vertex eingefügt. Erzeugen Sie neben der ersten eine zweite Verbindungslinie, so daß zwischen dem ersten und dem zweiten Raum ein Gang entsteht.



Sie haben nun also ein Gebilde aus drei geschlossenen Regionen - der erste Raum, der zweite und der kurze Gang dazwischen. Markieren Sie nun sämtliche Wände und weisen Sie ihnen wieder den Wandnamen **arc_wall** zu.

Nun scannen Sie die beiden neuen Räume ein (**[O]**) und weisen ihnen wieder jeweils die **dungeon**-Region zu. Diesmal wollen wir uns aber nicht mit den vordefinierten Default-Höhen begnügen, sondern wir geben dem Verbindungsgang eine Deckenhöhe von 8 Steps und dem zweiten Raum eine Bodenhöhe von -5 und eine Deckenhöhe von 20 Steps.

Auch die **border**-Region müssen wir wieder per Rechtsklick neu zuweisen (wieder mit **Default**-Höhen), da sich auch dort die Wände geändert haben. Räume, die verändert wurden und eine neue Zuweisung benötigen, werden vom WED übrigens rot dargestellt.

Wieder im Level (wenn unser Game noch im Hintergrund lief, müssen wir es zuerst mit **[F10]** beenden, um es dann mit **[Shift-^]** neu zu starten - nach dem Erzeugen neuer Regionen ist immer ein Neustart erforderlich) gähnt uns in einer Wand des ersten Raumes nun ein dunkler Verbindungsgang entgegen... Wenn nicht, haben Sie vermutlich vergessen, dem Gang oder dem zweiten Raum unterschiedlichen Höhen zu geben. Walls zwischen Regionen gleichen Namens und gleicher Höhe werden von der Engine nämlich als solide, undurchdringliche Wand dargestellt.

Dumm, daß wir nun unseren neuen Raum zwar betreten können, jedoch nicht mehr zurück kommen - die Bodenschwelle ist zu hoch! Warum definieren wir uns nicht auf die Schnelle eine Treppe?

Berühren Sie im Wall-Modus die eine Außenwand des Verbindungsganges, teilen Sie sie (**[S]**), teilen Sie jedes der Teilstücke noch einmal, und verfahren Sie ebenso mit der zweiten Außenwand. Jetzt bestehen die Wände aus jeweils vier Stücken. Verbinden Sie die im Gang gegenüberliegenden Vertices. Der Gang besteht nun aus vier querverlaufenden Regionen. Diesen müssen Sie jetzt nur noch die **dungeon**-Region mit jeweils einer um 1 Step unterschiedlichen Bodenhöhe zuweisen... schon ist die Treppe fertig!

Bei den 'höheren' Funktionen zum Definieren von Menüs, Actors, Waffen, Kampf- und Rätselaktionen sind Sie nun vorerst auf sich allein gestellt. Studieren Sie die Datei VRDEMO.WDL und lernen Sie daraus, wie Säulen, Portale, Lifte, Außenbereiche und Unterwasserregionen konstruiert werden! Die WDL ist eine mächtige und flexible Sprache, mit der Sie viel machen können, bis hin zur Implementation von künstlicher Pseudo-Intelligenz für Ihre Actors. Einige Beispiel-WDLs, die als Anregung dienen sollen, finden Sie auf der DEMO-Disk.

Dienstag: Die Player-Bewegung

Halt! Ganz wollen wir Sie doch noch nicht allein lassen. Als Beispiel für das Schreiben von WDL-Aktionen wollen wir heute eine der schwierigsten Aktionen überhaupt, nämlich die für die Bewegung des Players, erstellen. Die Move-Aktion enthält in geballter Form alle Tücken und Probleme, die Sie mit der WDL-Arithmetik erleben können. Es wird also ein bißchen mathematisch, doch keine Angst: Kenntnis der vier Grundrechenarten reicht aus. Am Schluß des Kapitels werden wir in der Lage sein, mit eigenen Aktionen dem Player ein beliebiges Bewegungsverhalten zu verleihen.

Die erste Move-Aktion

Wir benutzen wieder unser Minimal-Level. Nun jedoch wollen wir nicht auf die vorgefertigten Bewegungs-Aktionen in der **MOVE.WDL** zurückgreifen, sondern unsere eigenen schreiben. Da die Player-Bewegung permanent stattfinden soll, benutzen wir dafür eine unserer 16 globalen **EACH_TICK**-Aktionen. Diese 16 Aktionen werden automatisch nach jedem Bildaufbau ausgeführt. Wir werfen das vorgefertigte **set_walking** aus der **IF_START**-Action hinaus und setzen dafür unsere eigene Aktion ein:

```
ACTION my_move {  
    RULE    PLAYER_VX = FORCE_AHEAD;  
    RULE    PLAYER_VY = FORCE_STRAFE;  
    RULE    PLAYER_VROT = 0.1*FORCE_ROT;  
}  
  
ACTION my_start {  
    SET     EACH_TICK.16, my_move;  
}  
  
IF_START  my_start;
```

my_move ist die einfachste Form einer Bewegungs-Aktion: Joystick-Auslenkungen werden direkt in Player-Geschwindigkeiten umgesetzt. Die vordefinierten Skills **FORCE_AHEAD**, **FORCE_STRAFE** und **FORCE_ROT** enthalten Zahlenwerte, die unseren Eingaben mit Maus, Joystick oder Cursortas-

ten für die Vorwärts- und Seitwärtsbewegung und die Drehung des Players entsprechen. Bei Druck auf die [↑]-Taste z.B. nimmt **FORCE_AHEAD** den Wert **0,7** an, bei Druck auf die [↓]-Taste den Wert **-0,7**.

PLAYER_VX und **PLAYER_VY** sind ebenfalls vordefinierte Skills, mit denen man die Geschwindigkeit des Players in X- und Y-Richtung unseres Koordinatensystems angeben kann. Über **PLAYER_VROT** wird die Rotationsgeschwindigkeit des Players eingestellt. Die drei **RULE**-Anweisungen geben dem Player also einfach eine der Joystick-Auslenkung entsprechende Geschwindigkeit in Richtung der Koordinatenachsen bzw. eine Rotation (die Rotationseingabe haben wir vorher mit **0,1** multipliziert, da dem Player sonst schwindlig wird). Um eine Auslenkung von **FORCE_STRAFE** zu erreichen, benutzen wir die [↵] und [↶]-Tasten oder bewegen Joystick bzw. Maus seitwärts mit gedrückter [Alt]-Taste.

An der einfachen Bewegungs-Aktion fällt uns zunächst auf, daß die Bewegung nicht in Blickrichtung des Players erfolgt. Die Skills **PLAYER_VX** und **PLAYER_VY**, mit denen man die Geschwindigkeit des Players vorgibt, beziehen sich auf das gleiche X/Y-Koordinatensystem, in dem wir auch mit WED die Topographie erstellt haben. Eine Bewegung in Blickrichtung setzt sich aber aus einer Bewegung in X- und in Y-Richtung zusammen. Wir müssen also durch eine **RULE**-Anweisung dafür sorgen, daß sich eine Joystick-Auslenkung immer auf *beide* Skills - **PLAYER_VX** und **PLAYER_VY** - auswirkt, und zwar abhängig vom Blickwinkel **PLAYER_ANGLE**.

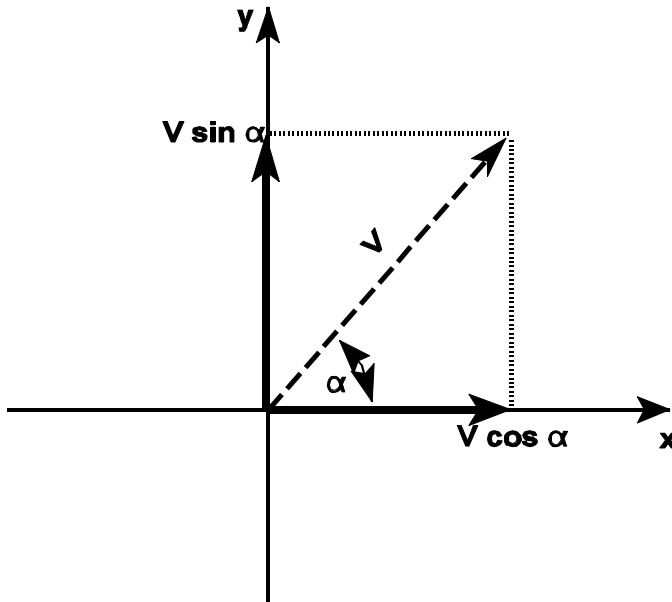


Abbildung 4 Sinus und Cosinus

Im Diagramm sieht man als dicke Pfeile die Verschiebungen, die in X- und Y-Richtung nötig sind, damit sich resultierend eine Verschiebung (gestrichelter Pfeil) in Richtung des Player-Blickwinkels α ergibt. Um den Player also um die Strecke V (Länge des gestrichelten Pfeils) in Richtung seines Blickwinkels zu bewegen, verschieben wir ihn um $V \cdot \cos \alpha$ in X-Richtung und zugleich um $V \cdot \sin \alpha$ in Y-Richtung.

Mit **PLAYER_SIN** und **PLAYER_COS** stehen uns zwei Skills zu Verfügung, die automatisch den Sinus und den Cosinus der Player-Blickrichtung enthalten. Damit sieht unsere Move-Aktion jetzt so aus:


```
ACTION my_move {  
    RULE    PLAYER_VX = FORCE_AHEAD * PLAYER_COS;  
    RULE    PLAYER_VY = FORCE_AHEAD * PLAYER_SIN;  
    RULE    PLAYER_VROT = 0.1*FORCE_ROT;  
}
```

Wenn wir die Seitwärtsbewegung hinzunehmen, wird aus den ersten beiden **RULEs**:

```
RULE    PLAYER_VX = FORCE_AHEAD * PLAYER_COS - FORCE_STRAFE * PLAYER_SIN;  
RULE    PLAYER_VY = FORCE_AHEAD * PLAYER_SIN + FORCE_STRAFE * PLAYER_COS;
```

Die letzten beiden **RULEs** stellen eine *Koordinatentransformation* dar. Wir transformieren unser um **PLAYER_ANGLE** gedrehtes Player-Koordinatensystem - repräsentiert durch die Skills **FORCE_AHEAD** und **FORCE_STRAFE** - in das 'ruhende' X/Y-Koordinatensystem, welches den Skills **PLAYER_VX** und **PLAYER_VY** zugrundeliegt.

Jetzt können wir den Player in alle Richtungen gleiten lassen. Leider aber wirkt die Bewegung noch unnatürlich und ruckartig, insbesondere wenn wir die Cursortasten dafür benutzen. Wir wollen uns jetzt überlegen, wie wir eine 'natürliche' Bewegung hinbekommen.

Beschleunigung, Trägheit, Reibung

Nehmen wir an, unser Player bewege sich mit einer konstanten Geschwindigkeit geradeaus. Die Wegstrecke, die er dabei in einer definierten Zeitspanne zurücklegt, ist umso größer, je größer die Geschwindigkeit und je größer die Zeitspanne ist:

$$s = v \cdot t$$

v = Geschwindigkeit in Steps pro Tick

s = Wegstrecke in Steps

t = Zeitspanne in Ticks

Strecken in unserer virtuellen Welt werden, wie wir uns erinnern, in Steps gemessen, und Zeiten in Ticks. Die Maßeinheit für Geschwindigkeiten ist folglich Steps/Tick. Kennen wir die Geschwindigkeit v , so können wir mit der obigen Formel zu jeder Zeit t den zurückgelegten Weg s ausrechnen.

Was passiert nun, wenn wir die Geschwindigkeit verändern wollen? Wir müssen dazu eine *Kraft* auf den Player einwirken lassen. Je stärker die Kraft ist, desto mehr verändert sich die Geschwindigkeit pro Zeiteinheit. Nun besitzt jeder Körper ein gewisses Beharrungsvermögen, einen Widerstand, den er solchen Veränderungen entgegensetzt. Dieser Widerstand - *Trägheit* genannt - resultiert aus der Masse des Körpers. Je größer die Masse ist, desto geringer fällt bei gleichem Kraftaufwand die Veränderung der Geschwindigkeit aus. Das können wir wieder in einer Formel ausdrücken:

$$a = \frac{K}{m}$$

a = Geschwindigkeitsänderung pro Tick (Beschleunigung)

K = Kraft

m = Masse

Es gibt drei Arten von Kräften, die wir bei der Beschleunigung in unserer virtuellen Welt berücksichtigen müssen. Wir haben es zunächst mit der *Antriebskraft* zu tun. Dies ist eine gewollte Geschwindigkeitsveränderung, die wir z.B. durch Auslenken des Joysticks erzeugen können. Die zweite Kraft-Art wollen wir *Drift* nennen. Beispiele dafür sind eine Strömung, die den Player in eine bestimmte Richtung treibt, oder die Schwerkraft, die ihn nach unten zieht. Driftkräfte können, wie im Fall der Strömung, von Region zu Region unterschiedlich sein.

Bei der dritten Kraft, die auf den Player einwirkt, handelt es sich um die *Reibung*. Diese Kraft versucht stets, seine Geschwindigkeit zu reduzieren. Sie ist - im Gegensatz zu den beiden anderen Kräften - um so größer, je schneller sich der Player bewegt:

$$R = -f \cdot m \cdot v$$

R = Reibungskraft
f = Reibungskoeffizient (Gleitreibung)
v = Geschwindigkeit
m = Masse

Durch das Minuszeichen soll ausgedrückt werden, daß diese Kraft der Geschwindigkeit entgegenwirkt. Der Reibungskoeffizient **f** ist abhängig von der Beschaffenheit des Bodens, auf dem sich der Player bewegt: Eis besitzt einen kleineren Reibungskoeffizienten als Stein. Auch die Masse **m** geht in die Gleichung ein, denn je schwerer der Player ist, desto stärker belastet er den Boden und desto größer ist die Reibung - vorausgesetzt, seine Schuhgröße bleibt konstant.

Alle drei Kräfte - Antrieb **K**, Drift **D** und Reibung **R** - addieren sich und wirken zusammen, um die Geschwindigkeit des Players zu verändern:

$$a = \frac{K+D+R}{m} = \frac{K+D}{m} - f \cdot v$$

Dies können wir direkt in eine entsprechende Aktion umsetzen. Für die Variablen in den Formeln müssen wir neue Skills definieren:

```

SKILL force_x    {}           // Kraft in X-Richtung
SKILL force_y    {}           // Kraft in Y-Richtung
SKILL drift_x    { VAL 0; }   // Drift in X-Richtung
SKILL drift_y    { VAL 0; }   // Drift in Y-Richtung
SKILL strength   { VAL 0.1; } // Kraft-Koeffizient
SKILL fric       { VAL 0.2; } // Reibungskoeffizient
SKILL mass       { VAL 1; }   // Träge Masse des Players

```

```

ACTION my_move {
  RULE force_x = strength*(FORCE_AHEAD*PLAYER_COS-FORCE_STRAFE*PLAYER_SIN);
  RULE force_y = strength*(FORCE_AHEAD*PLAYER_SIN+FORCE_STRAFE*PLAYER_COS);
  RULE PLAYER_VX = PLAYER_VX + (force_x+drift_x)/mass - fric*PLAYER_VX;
  RULE PLAYER_VY = PLAYER_VY + (force_y+drift_y)/mass - fric*PLAYER_VY;
  RULE PLAYER_VROT = 0.1*FORCE_ROT;
}

```

Aber was ist das? Mit unserer neuen Aktion kommt der Player überhaupt nicht mehr zum Stillstand! Er 'kriecht' auch bei losgelassenem Joystick mit Minimalgeschwindigkeit durch die Gegend. Sind unsere Gleichungen falsch?

Die Theorie stimmt. Leider haben wir es hier mit einem der Unterschiede zwischen Theorie und Praxis zu tun - sprich: Mit der begrenzten *Genauigkeit* unserer Arithmetik. Skills und **RULE**-Anweisungen werden bis auf drei Stellen hinter dem Komma genau berechnet. Weitere Nachkommastellen werden 'abgeschnitten'. Wird also ein Wert - auch ein Zwischenwert innerhalb einer **RULE** - kleiner als 0.001, so wird er automatisch als 0 behandelt. Dies aber passiert bereits, wenn **PLAYER_VX** den Absolutwert 0.005 unterschreitet. Multipliziert mit **fric**, also mit 0.2, ergibt sich dann nämlich für den Term (**fric** * **PLAYER_VX**) ein Wert kleiner als 0.001 - und damit 0.

Da **force_x** und **drift_x** ebenfalls 0 sind, verringert sich in der dritten **RULE**-Anweisung der Wert von **PLAYER_VX** nicht mehr. Der Player marschiert folglich im Kriechgang mit 0.005 Steps pro Tick ewig weiter. Mit solchen prinzipbedingten Tücken der Arithmetik müssen sich Software-Programmierer ständig herumschlagen, und manchmal auch Spielentwickler.

Die Lösung liegt in einer kleinen Umstellung unserer **RULEs**:

```
RULE  PLAYER_VX = (1-fric) * PLAYER_VX + (force_x+drift_x)/mass;  
RULE  PLAYER_VY = (1-fric) * PLAYER_VY + (force_y+drift_y)/mass;
```

Mathematisch völlig identisch mit der vorigen Version, aber jetzt funktioniert's! Wir können nun die Drift, die Stärke des Player (mit **strength**), seine Masse (mit **mass**) und die Reibung des Bodens (mit **fric**) regionsabhängig individuell einstellen. Auf Eis bekäme **fric** z.B. den Wert 0.05 und im Sumpf den Wert 0.5. Mit dieser neuen Move-Aktion bewegen wir uns jetzt außerdem viel 'flüssiger' durch die Gänge. Wir können sanft beschleunigen und wieder abbremsen. Dafür ist die Aktion mittlerweile gegenüber der ersten Version auch ziemlich angeschwollen. Sind wir nun fertig?

Leider noch nicht ganz. Wir verändern in unserer Aktion die Geschwindigkeiten in X- und Y-Richtung, indem wir die errechneten Beschleunigungen nach

jedem Bildzyklus einfach dazu addieren. Was passiert aber, wenn die Bildzyklen eine unterschiedlich lange Zeit dauern? Ganz klar: Auf schnellen PCs mit kurzen Bildzyklen beschleunigt der Player öfter um den gleichen Betrag und bewegt sich infolgedessen insgesamt schneller als auf langsameren Rechnern. Die Besitzer lahmere PCs werden also zusätzlich noch durch eine schneckenhafte Player-Bewegung gestraft.

Außerdem dürfen wir nicht vergessen, daß sich auch innerhalb unseres Games unterschiedlich lange Bildzyklen einstellen können. In ein und demselben Level kann die Frame-Rate bei unterschiedlichen Player-Positionen um bis zu dem Faktor drei schwanken. Wir brauchen also eine Art Ausgleichsfaktor - und der steht uns mit dem vordefinierten Skill **TIME_CORR** zur Verfügung. **TIME_CORR** hat den Wert 1, wenn die Frame-Rate genau 16 Bilder/sek beträgt, und verändert sich proportional zur Abweichung von diesem Mittelwert. Bei 8 Bildern/sek erreicht **TIME_CORR** also den Wert 2 und bei 32 Bildern/sek den Wert 0,5.

Mit Zeitausgleich sähe unsere **RULE** etwa so aus:

```
RULE PLAYER_VX = (1-TIME_CORR*fric) * PLAYER_VX +  
TIME_CORR*(force_x+drift_x)/mass;
```

Leider müssen wir hier wieder Sonderfälle berücksichtigen. Wenn z.B. das Produkt (**TIME_CORR** * **fric**) sich dem Wert 1 nähert, kommt der Player nicht mehr sauber zum Stillstand, sondern zappelt im Stand hin und her. Solche Phänomene kennt man unter dem Begriff *Regelschwingungen*. Wir sind hier an der Grenze dessen, was wir mit **RULEs** auf einigermaßen übersichtliche Weise berechnen können.

Die obige Formel zusammen mit Zeitausgleich, Kriech-Unterdrückung, Unterdrückung von Regelschwingungen und Korrektur aller Sonderfälle ist darum in der **ACCEL**-Anweisung im Programm selbst integriert. **ACCEL** beschleunigt eine Geschwindigkeit um einen angegebenen Betrag oder Skill, wobei die vordefinierten Skills **FRICTION** und **INERTIA** benutzt werden. Damit erhalten wir folgende Move-Aktion:

```
ACTION my_move {  
    SET    INERTIA, mass;  
    SET    FRICTION, fric;  
    RULE   force_x = strength * (FORCE_AHEAD * PLAYER_COS - FORCE_STRAFE * PLAY-  
        ER_SIN) + drift_x;  
    RULE   force_y = strength * (FORCE_AHEAD * PLAYER_SIN + FORCE_STRAFE * PLAY-  
        ER_COS) + drift_y;  
    ACCEL  PLAYER_VX, force_x;  
    ACCEL  PLAYER_VY, force_y;  
  
    SET    FRICTION, 0.5;  
    RULE   FORCE_ROT = 0.1 * FORCE_ROT;  
    ACCEL  PLAYER_VROT, FORCE_ROT;  
}
```

Wir haben die Drift direkt zu unseren Antriebskräften hinzu addiert und außerdem die **ACCEL**-Anweisung dazu benutzt, auch der Rotation ein besseres Verhalten zu verleihen. Damit ist unsere Move-Aktion fertig... für's erste. Der Player bewegt sich jetzt perfekt in X- und Y-Richtung. Aber was passiert, wenn er sich einer Treppenstufe nähert? Was ist mit den weiteren Freiheitsgraden, die den Player befähigen, auf- und abwärtszublicken, zu springen, zu tauchen und zu fallen?

Vertikale Bewegung

Während die horizontale Playerbewegung eigentlich immer gleich ist - abgesehen von Unterschieden beim Reibungskoeffizienten - unterscheidet sich die vertikale Move-Aktion grundlegend je nach der Art, in der der Player sich bewegt. Wir wollen in unseren Aktionen folgende Bewegungsarten implementieren:

Fahren: Hier müssen durch die Vertikalbewegung lediglich Höhenunterschiede der Regionen ausgeglichen werden, so daß der Player immer den etwa gleichen Abstand zum Boden hält.

Gehen: Hier bewegt sich der Player beim Voranschreiten rhythmisch auf und ab. Außerdem soll er springen und sich ducken können.

Beginnen wir mit dem Ausgleich der Höhenunterschiede. Hierfür sind die Skills **PLAYER_Z**, **PLAYER_SIZE**, **FLOOR_HGT** und **PLAYER_HGT** verantwortlich. Der Skill **PLAYER_HGT** gibt den Abstand der 'Füße' des Players vom Boden an. Dieser Abstand kann auch negativ werden, wenn der Player in den Boden 'einsinkt'.

Wir können es uns einfach machen, indem wir **PLAYER_Z** so berechnen, daß der Abstand der Augenhöhe des Players zum Boden immer genau dem **PLAYER_SIZE** entspricht:

```
RULE    PLAYER_Z = FLOOR_HGT + PLAYER_SIZE;
```

Wenn wir diese Anweisung in unsere Move-Aktion einfügen, können wir mit dem Player Regionen unterschiedlicher Bodenhöhe - z.B. Treppen - passieren. **PLAYER_HGT** hat jetzt immer den Wert 0. Leider aber erfolgt dadurch die Höhenanpassung ähnlich ruckartig und unnatürlich wie bei unserem ersten Versuch mit der Vorwärtsbewegung. Um eine weichere, realistische Vertikalbewegung hinzubekommen, müssen wir wieder die Kräfte betrachten, die vertikal auf den Player einwirken.

Solange sich der Player in der Luft befindet - d.h. **PLAYER_HGT** einen Wert größer 0 annimmt - wirkt auf ihn nur die *Schwerkraft* und die *Luftreibung*. Die Schwerkraft ist eine Drift, wie wir sie bereits kennengelernt haben; sie beschleunigt den Player nach unten...

...bis der Player den Boden berührt bzw. in ihn eindringt. Dann wirkt zusätzlich eine *elastische* Kraft. Dies ist eine neue Art von Kraft, die um so größer wird, je tiefer der Player in den Boden eingesunken ist. Sie beschleunigt ihn nach oben. Außerdem erhöht sich beim Eindringen in den Boden der vertikale Reibungskoeffizient beträchtlich.

Auch in harte Felsböden muß der Player einsinken können! In der Realität würde zwar der Boden nicht nachgeben, wohl aber die Kniegelenke des Players, was für den gleichen Effekt sorgt. Die elastische Kraft entspricht also der Elastizität seiner Gelenke oder - wenn er motorisiert ist - der Federung seines Fahrzeugs. Ebenso

können wir die erhöhte *Bodenreibung* auf die Reibungskraft seiner Muskeln oder die Stoßdämpfer seines Vehikels zurückführen.

```
SKILL force_z    {}           // Kraft in Z-Richtung
...
SKILL fric_air   { VAL 0.02; } // Luftreibungskoeffizient
SKILL fric_gnd   { VAL 0.97; } // Bodenreibungskoeffizient
SKILL gravity    { VAL -0.2; } // Gravitationskraft
SKILL knee_fac   { VAL 0.3; } // Elastizitäts-Koeffizient

ACTION my_move {
    ...           // die bisherigen Anweisungen,,,
    SET          FRICTION,fric_air;
    SET          force_z,gravity;
    IF (PLAYER_HGT<=0) { // Berührt Player den Boden?
        SET      FRICTION,fric_gnd;
        RULE     force_z = gravity - PLAYER_HGT*knee_fac;
    }
    ACCEL        PLAYER_VZ,force_z;
}
```

Mit der **IF**-Anweisung unterscheiden wir, ob sich der Player in der Luft oder am Boden befindet. Im letzten Fall wird **FRICTION** geändert und die elastische Kraft, die proportional der (negativen) Eindringtiefe **PLAYER_HGT** ist, zu **force_z** hinzu addiert.

Mit dieser neuen Move-Aktion können wir leicht ausprobieren, was geschieht, wenn wir die Bodenreibung vergessen. Entfernen Sie probeweise die Zeile **SET FRICTION,fric_gnd;** aus der Aktion und fahren Sie mit dem Player über eine Stufe! Den Effekt, der sich sogleich einstellen wird, kennen wir von Gummibällen oder Autos mit kaputten Stoßdämpfern. Den gleichen Effekt bekommen wir, wenn wir den **knee_fac** zu hoch wählen.

Damit haben wir den Fall des Fahrens abgehandelt. Beim Gehen kommt eine vertikale Auslenkung hinzu, die der Gehbewegung entspricht. Der Player wird gewissermaßen gestaucht oder gestreckt, je nachdem, in welcher Phase eines Schrittes er sich befindet. Diese streckenabhängige Auslenkung steht uns in Form des vordefinierten Skills **WALK** zur Verfügung. Wir benutzen diesen Skill einfach

zur Veränderung von **PLAYER_SIZE** und setzen die folgende Zeile ans Ende unserer Move-Aktion:

```
RULE    PLAYER_SIZE = my_size + 0.3 * WALK;
```

Der neue Skill **my_size** muß vorher mit **VAL 4** definiert werden; der Player ist also normalerweise 4 Steps hoch. Der Faktor **0.3** bestimmt die Stärke der Gehbewegung. Mit einer Umdefinition der vordefinierten Skills **WALK_PERIOD** und **WALK_TIME** können wir Schrittweite und Schwan-
kungszeit einstellen:

```
SKILL my_size      { VAL 4; }  
SKILL WALK_PERIOD  { VAL 5; }  
SKILL WALK_TIME    { VAL 10; }
```

Über den Skill **PLAYER_SIZE** können wir zusätzlich den Player mit der **[Ende]**-Taste sich ducken lassen. Dabei soll sich seine Höhe reduzieren. Zuerst die simple Variante (hinter der obigen **RULE PLAYER_SIZE...**-Zeile einfügen):

```
IF (FORCE_UP<0)  
{ RULE PLAYER_SIZE = PLAYER_SIZE - 1.5; }
```

Der Skill **FORCE_UP** nimmt einen Wert von **-0,7** an, wenn **[Ende]** gedrückt wird. Besser wirkt es, wenn das Ducken und Wiederaufrichten fließend erfolgt. Hierzu brauchen wir einen neuen Skill:

```
SKILL    duck_val  { VAL 0; }  
  
...  
RULE     duck_val = 0.8*duck_val;    // Reduzieren zum 'Ausklingen'  
IF {FORCE_UP<0}  
{ RULE  duck_val = duck_val + 0.5*FORCE_UP; }  
RULE     PLAYER_SIZE = PLAYER_SIZE + duck_val;  
...  

```

Um diese Aktion zu verstehen, muß man sich vergegenwärtigen, daß sie ständig - jeden Tick - wiederholt wird. Der Wert von **duck_val** ist normalerweise 0, so daß sich **PLAYER_SIZE** nicht verändert. Wenn wir nun die **[Ende]**-Taste betätigen,

erhält **FORCE_UP** einen negativen Wert, so daß die auf **IF** folgende Anweisung ausgeführt wird. Hiermit wird von **duck_val** ein Betrag subtrahiert, was in der darauffolgenden Zeile zur Verringerung der Player-Größe führt. Das Multiplizieren mit **0.8** in der ersten **RULE**-Anweisung sorgt dafür, daß der Skill **duck_val** von selbst immer wieder der **0** zustrebt, und zwar um so mehr, je höher sein Absolutwert ist.

Warum haben wir hier nicht einen Skill **duck_speed** eingeführt und unsere gewohnte **ACCEL**-Anweisung benutzt? Weil Ducken in der Realität nichts mit Beschleunigung zu tun hat. Man sieht sofort, daß das Multiplizieren mit **0.8** einen natürlicheren Effekt ergibt: Das Ducken und Wiederaufrichten klingt sanfter aus. Theoretisch müßten wir übrigens auch hier noch eine Zeitkorrektur anbringen, aber man kann's auch übertreiben...

Wie tief duckt sich übrigens der Player mit der obigen Aktion? Das ist offensichtlich von unseren Faktoren **0.8**, **0.5** und dem **FORCE_UP**-Wert bei Betätigung der **[Ende]**-Taste (normalerweise **-0.7**) abhängig. Das Ducken ist beendet, wenn die Reduzierung von **duck_val** durch Multiplikation mit **0.8** den negativen Zuwachs durch **0.5*FORCE_UP** gerade wieder aufhebt:

$$(1 - 0.8) \cdot \text{duck_minimum} = 0.5 \cdot 0.7 \quad \Rightarrow \quad \text{duck_minimum} = \underline{\underline{1.75}}$$

Was noch fehlt, ist das Springen. Wir könnten dem Player beim Druck auf die **[Pos1]**-Taste - d.h. bei positiver **FORCE_UP** - einfach eine feste Beschleunigung nach oben geben. In der Realität besteht ein Sprung allerdings aus einem Ducken mit anschließendem Hochschnellen, also aus zwei 'gegenläufigen' Bewegungen. Wie können wir einen solchen Ablauf in unsere Move-Aktion integrieren?

Ein Sprung soll aus drei aufeinanderfolgenden Phasen bestehen - der Phase des Duckens, der Beschleunigung nach oben und einem antriebslosen Zustand, in dem sich der Player in der Luft befindet und ganz der Schwerkraft ausgeliefert ist. Um zwischen diesen Sprungphasen unterscheiden zu können, führen wir den neuen Skill **jump_phase** ein. Dieser Skill steht normalerweise auf 0, d.h. es ist kein Sprung im Gange. Durch Druck auf die **[Pos1]**-Taste soll der Skill auf 1 gesetzt werden und dann die gerade ablaufende Sprungphase anzeigen. In der ersten Phase

(**jump_phase** = 1) soll sich der Player ducken, in der folgenden (2) soll er hochschnellen. Es folgt eine Phase (3) des freien Flugs. In dem Moment, in dem er nach dem Sprung wieder den Boden berührt, soll **jump_phase** wieder auf 0 gesetzt werden, so daß bei erneutem Tastendruck ein neuer Sprung beginnen kann.

```
SKILL jump_phase { VAL 0; }
```

```
...
IF (jump_phase>0)          // Sprung im Gange?
{ GOTO jump_1; }
IF (FORCE_UP<0.1)          // KEIN Tastendruck auf Pos1?
{ GOTO no_jump; }
SET      jump_phase,1;      // Sprung beginnt
jump_1:
IF (jump_phase>1)
{ GOTO jump_2; }
RULE     duck_val = duck_val - 0.5;      // Ducken
IF (duck_val<-0.7) // Tief genug geduckt? (Minimum beachten!)
{ SET     jump_phase,2; } // Dann jetzt hochspringen
GOTO     no_jump;
jump_2:
IF (jump_phase>2)
{ GOTO jump_3; }
SET      duck_val,0;        // Ducken beenden
RULE     PLAYER_Z = FOOT_HGT + my_size; // Wieder aufrichten
SET      PLAYER_VZ,0.5;     // Absprung-Geschwindigkeit
SET      jump_phase,3;      // Als naechstes: Freier Flug
GOTO     no_jump;
jump_3:
IF ((PLAYER_VZ<=0) && (PLAYER_HGT<=0) // Player gelandet?
{ SET     jump_phase,0; } // Sprung zu Ende
no_jump:
...                // Hier geht's weiter mit RULE duck_val...
```

Diesen Abschnitt müssen wir vor der **PLAYER_SIZE**-Reduzierung via **duck_val** in die Aktion einfügen, sonst kann das Ducken nicht funktionieren.

Nun brauchen wir noch die Aktion, um den Player aufwärts oder abwärts blicken zu lassen. Hierzu dient der Skill **PLAYER_TILT**. Das vertikale Schwenken des

Blickwinkels soll ähnlich erfolgen wie das Ducken, d.h. der Blickwinkel soll nach Loslassen der Taste wieder in die Ausgangsstellung zurückkehren:

```
RULE    PLAYER_TILT = 0.8 * PLAYER_TILT + 0.3 * FORCE_TILT;
```

Wir können den Blickwinkel aber noch für ein paar Effekte mehr benutzen. Zum Beispiel könnten wir den Skill **ACCELERATION** auswerten, um den Player bei der Beschleunigung nach hinten und beim Abbremsen nach vorne kippen zu lassen. Das macht sich in Rennspielen sehr gut. Hier wollen wir jedoch einen anderen Effekt einbauen: Wenn der Player von einer Klippe stürzt - z.B. von der Säulengalerie im VRDEMO - soll er eine halbe Drehung nach vorne beschreiben, so daß er den Boden auf dramatische Weise direkt auf sich zukommen sieht. Das darf allerdings nicht passieren, wenn er gerade im Sprung begriffen ist:

```
IF (jump_phase<=0)    // kein Sprung?  
{    RULE    PLAYER_TILT = PLAYER_TILT + 0.03 * (PLAYER_HGT + 0.3); }
```

Wir haben den Wert 0.3 zu **PLAYER_HGT** hinzu addiert, weil der Player normalerweise aufgrund der Schwerkraft und der Boden-Elastizität immer ein Stück in den Boden einsinkt. Die Blickwinkelveränderungen haben wir übrigens wieder ohne Zeitkorrektur durchgeführt, um es nicht zu kompliziert zu machen...

Unsere komplette Move-Aktion mit allen Features ist damit ziemlich länglich geworden. Sie ist am Ende des nachfolgenden Kapitels zusammen mit den dort behandelten Zusatzaktionen für Unterwasser-Regionen in ihrer ganzen Pracht noch einmal abgedruckt.

Noch eine Überlegung zum Schluß: Wie können wir eigentlich erreichen, daß Skills wie **fric** oder **gravity** sich abhängig von der Region, in der sich der Player befindet, ändern? Es gibt hierzu mehrere Möglichkeiten. Wir könnten die acht 'universellen' Skills der Region benutzen und festlegen, daß z.B. **SKILL5** der Reibung, **SKILL6** der Gravitation entsprechen soll. Dann brauchen wir nur unsere Move-Aktion mit den folgenden Zeilen beginnen zu lassen:

```
ACTION my_move {  
    SET    fric, HERE.SKILL5;  
    SET    gravity, HERE.SKILL6;  
    ...  
}
```

HERE ist das Synonym für den Namen der Region, in der sich der Player gerade befindet. Leider müßten wir auf diese Weise aber in jeder Regionsdefinition den Parametern **SKILL5** und **SKILL6** explizit Werte zuweisen. Außerdem wären diese Region-Skills dann 'verbraucht'. Eine bessere Methode ist, nur in solchen Regionen, in denen z.B. die Bodenreibung vom Normwert abweicht, eine **IF_ENTER**- und **IF_LEAVE**-Aktion zu definieren. Wir erinnern uns, daß **IF_ENTER** beim Betreten und **IF_LEAVE** beim Verlassen einer Region durchgeführt wird:

```
ACTION set_sumpf_fric  { SET fric, 0.5; }  
ACTION set_normal_fric { SET fric, 0.2; }  
  
REGION sumpf {  
    ...           // restliche Regions-Parameter...  
    IF_ENTER     set_sumpf_fric;  
    IF_LEAVE     set_normal_fric;  
}
```

Special effects

Wir bauen nun ein paar kleine Spezialeffekte ein, die sich in Games immer wieder gut machen, weil sie den Spieler verblüffen. Zum Auftakt geben wir dem Player eine Brille mit Gummilinsen, mit denen er über die Tasten **[Einfg]** und **[Entf]** seinen Blickwinkel zoomen kann.

```
ACTION my_move {  
    ...           // der ganze Rest...  
    IF (KEY_INS!=0) // Taste Einfg gedrückt?  
    { RULE  PLAYER_ARC = PLAYER_ARC - 0.1; }  
    IF (KEY_DEL!=0) // Taste Entf gedrückt?  
    { RULE  PLAYER_ARC = PLAYER_ARC + 0.1; }  
}
```

Als nächsten *special effect* wollen wir uns ein Erdbeben vornehmen. So etwas wirkt in Filmen und auch in Computerspielen immer sehr eindrucksvoll. Das Erdbeben soll mit der **[E]**-Taste ausgelöst werden. Wir steuern diesen Spezialeffekt mit einer neuen Aktion, die parallel zu unserer Move-Aktion ausgeführt wird:

```
SKILL richter      { VAL 0; }    // Richter-Skala

ACTION my_move {
    ...           // der ganze Rest...
    RULE  PLAYER_X = PLAYER_X + richter*(RANDOM(1) - 0.5);
    RULE  PLAYER_Y = PLAYER_Y + richter*(RANDOM(1) - 0.5);
    RULE  PLAYER_Z = PLAYER_Z + 2*richter*(RANDOM(1) - 0.5);
}

ACTION quake {
    SET    richter,0.1;           // Erdbeben langsam beginnen...
    WAITT  16;                   // 16 Ticks = 1 Sekunde
    SET    richter,0.3;
    WAITT  32;
    SET    richter,0.5;           // Höhepunkt
    WAITT  48;
    SET    richter,0.2;           // Verschnaufpause...
    WAITT  80;
    SET    richter,0.6;           // Zweiter Höhepunkt
    WAITT  32;
    SET    richter,0.1;           // Ausklingen
    WAITT  16;
    SET    richter,0;             // wieder Ruhe
}

IF_E quake;
```

Um den Player effektiv hin- und herzuschütteln, versetzen wir ihn mit den 3 **RULE**-Anweisungen um einen Zufallswert in irgendeine Raumrichtung. Die Stärke der Versetzung wird durch unseren neuen Skill **richter** vorgegeben. Durch das Subtrahieren von **0.5** erreichen wir, daß der betreffende Zufallsterm einen Wert zwischen -0.5 und +0.5 annimmt (der **RANDOM**-Wertebereich liegt zwischen 0 und 1).

In der Aktion **quake** wird die **WAITT**-Anweisung eingesetzt, um den Skill **richter** in einem bestimmten Zeitablauf an- und abschwellen zu lassen. **WAITT** läßt einfach die Aktion für die angegebene Anzahl von Ticks (1/16 Sekunden) 'schlafen'.

Zuguterletzt wollen wir eine stillschweigende Einschränkung unserer bisherigen Move-Aktionen fallen lassen: Wer sagt denn, daß sich dabei immer der Player bewegen muß? Wir könnten genausogut mit Maus oder Joystick eine Player-Figur als Actor steuern oder - umgekehrt - unseren Blick einem Actor leihen. Diese letzte Möglichkeit wollen wir realisieren. Hierzu benutzen wir die VRDEMO.WDL, in der unser kugelförmiger 'Guide' herumschwebt. Wir wollen die Welt einmal mit den Augen des Guide betrachten. Der ganze Blickwechsel soll aber erst durch Berühren des Guide, d.h. durch einen **IF_NEAR**-Event ausgelöst werden.

```
ACTION guide_eye {
    SET    PLAYER_X,MY.X;
    SET    PLAYER_Y,MY.Y;
    // relative Höhe absolut machen
    RULE    PLAYER_Z = MY.HEIGHT + THERE.FLOOR_HGT;
    SET     HERE,THERE;           // Player-Region setzen

    SET     FRICTION,0.5;
    RULE    FORCE_ROT = 0.1 * FORCE_ROT;
    ACCEL    PLAYER_VROT, FORCE_ROT; // Rotieren wollen wir noch selbst
}

ACTION change_body {
    SET     EACH_TICK.16,NULL;    // alte Move-Aktion löschen
    SET     MY.EACH_TICK,guide_eye; // neue starten
    SET     MY.PASSABLE,1;        // sonst prallt Player von sich selbst ab
    SET     MY.TEXTURE,blackpixel_tex;
}

ACTOR kugel {
    ...
    DIST      0;
    FLAGS     CAREFULLY;
    IF_NEAR   guide_eye;
}
```

Das bisher vorgegebene **BERKELEY**-Flag des Actors darf nicht länger gesetzt sein, sonst würde er sich nie bewegen, da er sich ja selbst nicht sieht. Ebenso haben wir seine **DIST** auf 0 gesetzt, um eine **IF_NEAR**-Aktion erst bei Berührung der Kugel auszulösen. Die neue Textur **blackpixel_tex** definieren wir uns vorher aus einer kleinen Bitmap in der transparenten Farbe 0, damit uns die Actor-Textur selbst nicht ständig wie ein Brett vor dem Kopf hängt.

Und eine Kleinigkeit müssen wir noch am **WAY** des Actors verändern, denn auf seinem normalen Weg durch den Level wird er jetzt in eine 'Falle' tappen! Mit dieser Aufgabe wollen wir Sie aber nun allein lassen. Es wird für Sie jetzt sicherlich ein leichtes sein, weitere Bewegungs-Aktionen für das Schwimmen, Tauchen, Klettern, Skifahren und für ferngesteuerte Kameras zu schreiben. Viel Spaß!

Mittwoch: Portale, Lifte, Wasserfälle

Bisher haben wir uns darauf beschränkt, *statische* Welten zu konstruieren. Doch ein 3D-Spiel wird wesentlich effektvoller, wenn sich in unserer Welt etwas bewegt - wenn sich auf Knopfdruck Räderwerke in Bewegung setzen, Geheimportale öffnen oder sich der Player plötzlich unter Wasser wiederfindet! Heute wollen wir daher eine Bibliothek von universell einsetzbaren WDL-Aktionen erstellen, mit denen wir Teile unserer Welt auf unterschiedliche Art in Bewegung versetzen können.

Portale

Das einfachste und zugleich am häufigsten benutzte Beispiel für Bewegung in unserer Welt ist eine Tür, die sich öffnet. Und am einfachsten läßt sich das Öffnen einer Tür durch eine *Texturanimation* realisieren. Beginnen wir also damit: Wir wollen eine Wand definieren, die bei Annäherung des Players einen Animationszyklus durchläuft und beim Entfernen des Players die Animation rückwärts abspielt. In der Endphase der Animation kann der Player die Wand durchschreiten.

Als Beispiel für eine solche texturanimierte Tür definieren wir uns ein vierflügeliges Pneumatik-Schott. Die Bitmaps dazu müssen Sie allerdings leider selbst zeichnen und die Sounds selbst sampeln. Zuerst definieren wir Sound, Bitmaps und Texturen für die Tür sowie die Tür-Wand selbst:

```
SOUND tuer_zisch, <tuer_auf.wav>;
```

```
BMAP gtuer_map, <grauxit2.lbm>,0,0,256,256; // geschlossene Tür  
BMAP gtuer_auf1, <grauxit3.lbm>,0,0,256,256; // 6 Phasen für öffnende Tür  
BMAP gtuer_auf2, <grauxit4.lbm>,0,0,256,256;  
BMAP gtuer_auf3, <grauxit5.lbm>,0,0,256,256;  
BMAP gtuer_auf4, <grauxit6.lbm>,0,0,256,256;  
BMAP gtuer_auf5, <grauxit7.lbm>,0,0,256,256;  
BMAP gtuer_auf6, <grauxit8.lbm>,0,0,256,256;
```

```
////////////////////////////////////
```

```

TEXTURE grautuer_auftex {
    SCALE_XY  18,18;
    CYCLES     7;
    BMAPS      gtuer_map,gtuer_auf1,gtuer_auf2,gtuer_auf3,gtuer_auf4,
               gtuer_auf5,gtuer_auf6;
    DELAY      2,2,1,1,1,2,2;
    FLAGS      ONESHOT;
    SOUND      tuer_zisch;
    SCYCLE      2;
    SVOL        0.3;
    SDIST       50;
}

TEXTURE grautuer_zutex {
    SCALE_XY  18,18;
    CYCLES     6;
    BMAPS      gtuer_auf5,gtuer_auf4,gtuer_auf3,gtuer_auf2,
               gtuer_auf1,gtuer_map;
    DELAY      2,1,1,1,1,2,2;
    FLAGS      ONESHOT;
    SOUND      tuer_zisch;
    SCYCLE      1;
    SVOL        0.2;
    SDIST       50;
}

////////////////////////////////////////
WALL grautuer {
    TEXTURE    grautuer_auftex;
    FLAGS      SAVE;
    DIST       22;
    IF_NEAR    grautuer_auf;
    IF_FAR     grautuer_zu;
}

```

Da die Richtung der Animation festgelegt ist, benötigen wir zwei Texturen für die sich öffnende und schließende Tür. Beide Texturen unterscheiden sich im wesentlichen nur in der Reihenfolge der Animationsphasen. Damit die Animationen zum Öffnen und Schließen der Tür nur jeweils einmal ablaufen, benutzen wir das **ONESHOT**-Flag. Bei den **DELAY**-Zeiten haben wir berücksichtigt, daß die

Türflügel am Anfang ihrer Bewegung beschleunigt und am Ende abgebremst werden; in der Mitte bewegen sie sich am schnellsten.

Etwas Neues finden wir bei der Türdefinition: Wir haben hier zwei *Events* definiert, **IF_NEAR** und **IF_FAR**, die *Aktionen* auslösen können. Mit solchen Events werden wir es jetzt noch öfter zu tun bekommen. Im Referenzkapitel lesen wir nach, daß die **IF_NEAR**-Aktion bei Annäherung des Players über die angegebene **DIST**-Grenze hinaus ausgelöst wird; bei der **IF_FAR**-Aktion verhält es sich umgekehrt. Das Flag **SAVE** muß gesetzt werden, damit beim Speichern und Laden von Spielständen der Zustand der Tür rekonstruiert werden kann, denn unsere Aktionen verändern die Tür-Textur:

```
ACTION grautuer_auf {
    SET    MY.TEXTURE,grautuer_auftex;
    CALL    texdoor_open;
}

ACTION grautuer_zu {
    SET    MY.TEXTURE,grautuer_zutex;
    CALL    texdoor_close;
}

////////////////////////////////////
ACTION texdoor_open {
    SET    MY.DIST,22;        // Hysterese
    SET    MY.TRANSPARENT,1;
    SET    MY.PLAY,1;
    SET    MY.EACH_CYCLE,texdoor_checkopen;
}

ACTION texdoor_checkopen {
    SET    MY.PASSABLE,1;    // Jetzt kann Player durch die Tür!
}

ACTION texdoor_close {
    SET    MY.DIST,20;
    SET    MY.PASSABLE,0;    // Tür wieder versperren
    SET    MY.PLAY,1;
    SET    MY.EACH_CYCLE,texdoor_checkclose;
}
```

```
ACTION texdoor_checkclose {  
    SET      MY.TRANSPARENT,0;  
}
```

Zunächst fällt auf, daß wir die Aktionen scheinbar grundlos aufgeteilt haben: **grautuer_auf** benutzt **texdoor_open**. Es erweist sich bei der Entwicklung größerer Spiele als nützlich, Aktionen in eine *spezifische* und eine *allgemeine* Aktion aufzuteilen. Wir können die allgemeine Aktion **texdoor_open** für sämtliche Türen benutzen, die per Texturanimation geöffnet werden. Die Aktion **grautuer_auf** hingegen bezieht sich nur auf spezielle Türen mit den Texturen **grautuer_auf/zutex**. Wenn unser Spiel aus mehreren Leveln besteht, empfiehlt es sich, allgemeine Aktionen wie **texdoor_open** und **texdoor_close** in eine separate WDL-Datei auszulagern, die dann per **INCLUDE**-Schlüsselwort in die einzelnen Level-WDL-Dateien eingefügt wird.

Da die Aktionen von Wand-Events ausgelöst werden, können wir das Synonym **MY** zur Bezeichnung der auslösenden Wand verwenden. Bei Annäherung an die Tür wechselt **grautuer_auf** zu der Textur mit den Öffnungsphasen und startet die Animation mit dem Flag **PLAY**. Zuvor wird **TRANSPARENT** gesetzt, damit man durch den Spalt der halboffenen Tür bereits hindurchblicken kann. Zugleich mit dem Beginn der Texturanimation wird eine **EACH_CYCLE**-Aktion definiert, welche beim Erreichen der letzten Phase - bei vollständig geöffneter Tür - ausgelöst wird. Diese Aktion setzt einfach die Wand auf **PASSABLE**, damit sie vom Player durchschritten werden kann.

Ein kleiner Trick - genannt 'Hysteresis' - ist die Erhöhung der **DIST** auf 27 Steps. Die **IF_FAR**-Aktion würde also jetzt erst etwas später ausgelöst. Hiermit erreichen wir, daß die Tür - wenn der Player sich gerade in der kritischen Entfernung hin- und herbewegt - sich nicht dauernd öffnet und wieder schließt.

Hat der Player wieder eine kritische Entfernung zur Tür erreicht, wird **grautuer_zu** ausgelöst. Sinngemäß passiert hier das umgekehrte wie bei der Aktion zum Türöffnen. Wir dürfen nicht vergessen, die **DIST** auf den ursprünglichen Wert zurückzusetzen. Nach Ablauf der Phasen wird auch **TRANSPARENT** zurückgesetzt. Zwar enthält die letzte Texturphase **gtuer_map** keine transparenten Teile,

trotzdem würde - wenn wir das **TRANSPARENT**-Flag gesetzt ließen - der Raum hinter der Tür beim Bildaufbau mitberücksichtigt werden, was Rechenzeit kostet. Das **TRANSPARENT**-Flag sollte also nur verwendet werden, wenn es unbedingt nötig ist!

Wenn wir die Tür als Wand im Level plazieren, muß sich natürlich auf beiden Seiten die gleiche Region befinden, sonst funktioniert das Durchschreiten mit **PASSABLE** nicht. Noch ein Hinweis: Beim Konstruieren von Wänden mit **IF_NEAR**- oder **IF_FAR**-Aktionen dürfen wir nicht vergessen, daß sich die kritische Entfernung **DIST** nur auf den Abstand zum nächstgelegenen Vertex der Wand bezieht. Wenn eine Wand 50 Steps lang ist und eine kritische Distanz von 20 Steps hat, könnte der Player die Mitte der Wand berühren, ohne einen Event auszulösen!

Lifte

In 3D-Games sind vertikale Lifte ein beliebtes Mittel, um dem Player neue Blickwinkel zu eröffnen. Sie sind vor allen Dingen leicht zu realisieren. Wir müssen lediglich in einer **EACH_TICK**-Aktion den Boden einer Region wie einen Lift stetig heben oder senken, bis eine bestimmte Zielhöhe erreicht ist. Dafür, daß der Player mit dem Lift mitfährt, sorgt unsere Move-Aktion für die Playerbewegung.

Damit unsere Lift-Aktionen möglichst allgemein einsetzbar sind, verlagern wir ihre spezifischen Eigenschaften, nämlich Geschwindigkeit, Ausgangs- und Zielhöhe, in festgelegte Region-Parameter der betreffenden Lift-Region. Es gibt acht solcher frei verwendbaren Parameter, die über die Schlüsselworte **SKILL1...SKILL8** mit Zahlen belegt und in Aktionen verändert und ausgewertet werden können. Einen Lift könnte man dann beispielsweise so definieren:

```
REGION lift0_20 {  
    FLOOR_HGT  0;  
    CEIL_HGT   40;  
    FLOOR_TEX  boden_tex;  
    CEIL_TEX   decke_tex;  
    FLAGS      SAVE;
```

```

SKILL3    0;           // Untere Lift-Höhe in Steps
SKILL4    20;          // Obere Lift-Höhe
SKILL6    0.3;         // Liftgeschwindigkeit in Steps pro Tick
IF_ENTER  lift_start;
}

```

Das Flag **SAVE** ist auch hier wichtig, da sich die Region während des Spiels verändern wird; beim Abspeichern der Spielstände soll die Liftposition mitgespeichert werden. Unser Lift soll automatisch von Höhe 0 auf Höhe 20 fahren oder umgekehrt, je nachdem, von welcher Höhe aus der Player ihn betreten hat. Hierzu müssen unsere verwendeten Regions-Parameter **SKILL3**, **SKILL4** und **SKILL6** in den folgenden Aktionen ausgewertet werden:

```

SOUND rumpel,<rumpel.wav>; // sehr kurzer Sound (<0.3 sec!)

```

```

SKILL lift_speed { }           // Zwischenwert

```

```

ACTION lift_start {
    IF (THERE.FLOOR_HGT<THERE.SKILL4) // rauf oder runter?
    { SET     THERE.EACH_TICK, lift_uptick; }
    ELSE
    { SET     THERE.EACH_TICK, lift_downtick; }
}

```

```

ACTION lift_uptick {
    SET     lift_speed, THERE.SKILL6;
    RULE     lift_speed = TIME_CORR*lift_speed;
    RULE     THERE.FLOOR_HGT += lift_speed;
    IF (THERE==HERE)           // Steckt Player im Lift?
    { RULE    PLAYER_Z += 0.5*lift_speed; }

    IF (THERE.VISIBLE)         // Wenn Lift sichtbar,
    { SET     RENDER_MODE,1; }  // Bild staendig Neuberechnen
    PLAY_SOUND    rumpel,0.3;

    IF (THERE.FLOOR_HGT>=THERE.SKILL4) { // Ziel-Höhe erreicht
        SET     THERE.FLOOR_HGT, THERE.SKILL4; // Sollwert
        SET     THERE.EACH_TICK, NULL;
    }
}

```

```
ACTION lift_downtick {
    SET      lift_speed, THERE.SKILL6;
    RULE      lift_speed = -TIME_CORR*lift_speed;
    RULE      THERE.FLOOR_HGT += lift_speed;
    IF (THERE==HERE)          // Steckt Player im Lift?
    // Ruckeln unterdrücken
    { RULE    PLAYER_Z += 0.8*lift_speed; }

    IF (THERE.VISIBLE)
    { SET      RENDER_MODE,1; }
    PLAY_SOUND    rumpel,0.3;

    IF (THERE.FLOOR_HGT<=THERE.SKILL3) {
        SET      THERE.FLOOR_HGT, THERE.SKILL3;
        SET      THERE.EACH_TICK, NULL;
    }
}
```

Da die Aktionen direkt vom **IF_ENTER**-Event der Region ausgelöst wurden, können wir für die Region selbst das Synonym **THERE** verwenden. Die Aktion **lift_start** prüft anhand der aktuellen Höhe der Region, ob der Lift unten oder oben steht und ob er folglich aufwärts oder abwärts fahren soll. Das Bewegen der Region wird dann von der **EACH_TICK**-Aktion **lift_uptick** bzw. **lift_downtick** erledigt.

In den beiden fast identischen **EACH_TICK**-Aktionen wird zuerst aus der Liftgeschwindigkeit **THERE.SKILL6** und dem Korrekturfaktor **TIME_CORR** die zeitkorrigierte Geschwindigkeit **lift_speed** gebildet. Diese wird - gegebenenfalls negativ - auf die Bodenhöhe des Lifts aufaddiert. Auf diese Weise bewegt sich die Region ein Stückchen aufwärts bzw. abwärts.

Nun folgen zwei etwas merkwürdige Zeilen. Wenn sich der Player im Lift befindet, d.h. die Lift-Region **THERE** mit der Player-Region **HERE** übereinstimmt, wird seine vertikale Position **PLAYER_Z** verändert. Warum? Unsere Move-Aktion sorgt dafür, daß der Player immer etwa den gleichen Abstand zum Boden hält. Doch wenn sich der Boden selbst bewegt, ergeben sich vor allem bei Bewegungen nach unten durch das 'Nachfedern' des Players unschöne Ruckel-effekte. Durch das teilweise Mitbewegen von **PLAYER_Z** mit dem Liftboden wird das Ruckeln unterdrückt.

Wenn die Lift-Region sichtbar ist, wird der Skill **RENDER_MODE** auf 1 gesetzt, damit das Bild gegebenenfalls auch dann neu aufgebaut wird, wenn sich der Player nicht bewegt. Außerdem wird das Liftgeräusch **rumpel** ausgelöst. Da dieses Geräusch jeden Tick von neuem abgespielt wird, sollte es sehr kurz sein - höchstens 1/3 Sekunde -, damit sich die einzelnen Liftgeräusche nicht zu stark überlappen.

Am Schluß der Aktion wird geprüft, ob der Lift seine Endposition - **THERE.SKILL4** bzw. **THERE.SKILL3** - bereits überschritten hat. In diesem Falle wird der Boden noch einmal genau auf die Endposition gesetzt und die Aktion beendet. Übrigens: Die Anweisung **END** beendet eine **EACH_TICK**-Aktion natürlich nicht endgültig, sondern läßt sie lediglich beim nächsten Tick wieder von vorne beginnen! Die Aktion wird erst dann nicht mehr ausgeführt, wenn der betreffende **EACH_TICK**-Event auf **NULL** gesetzt wird.

Auf die gleiche Weise wie Lifte könnten wir natürlich auch vertikale Portale oder Fallgatter definieren. Hier würde einfach die **CEIL_HGT** bewegt. Es gibt aber einen noch einen anderen Weg, Türen in unser Level zu implementieren...

Flügeltüren

Statt Türen senkrecht zu bewegen, können wir sie auch, wie wir es aus dem wahren Leben gewohnt sind, horizontal auf- oder zuschwingen lassen. Das ist auch nicht viel komplizierter zu realisieren als eine Texturanimation. Zuerst definieren wir die Tür als einzeln stehende schmale rechteckige Region. Keine der vier Wände dieser Region darf mit einer anderen Wand verbunden sein, sonst könnte sie die Tür-Region beim Öffnen oder Schließen in diese hineindrehen.

Außer den vier Wänden brauchen wir noch eine Türangel. Um diese herum wird sich die Tür-Region dann drehen. Als Türangel dient uns ein unsichtbares **THING**, welches wir knapp an einer Schmalseite der Tür innerhalb der Region platzieren. Bei der Positionierung der Tür-Region per WED sollten wir darauf achten, daß der Öffnungsbereich frei bleibt - die Tür darf keine Regionsgrenzen überschreiten!


```
THING tuer_angel {          // muss innerhalb der Region liegen!
    TEXTURE    dummy_tex;
    DIST       15;
    IF_NEAR    swing_open;
    IF_FAR     swing_close;
}
REGION holz_tuer {
    FLOOR_TEX  holz_tex;
    CEIL_TEX   holz_tex;
    FLOOR_HGT  10;
    CEIL_HGT   10;
    FLAGS      SAVE;
    SKILL3     0;          // Anfangswinkel bei geschlossener Tür (0..3.14)
    SKILL4     1.57;       // Endwinkel bei offener Tür (1.57 = 90°)
    SKILL5     0;          // Aktueller Türwinkel
    SKILL6     0.1;        // Türgeschwindigkeit in Radiant pro Tick
    GENIUS     tuer_angel;
}
```

Auch alle Wände der Tür-Region müssen, da sie sich verändern, das SAVE-Flag erhalten. Durch die Vorgabe des Anfangs- und Endwinkels in **SKILL3** und **SKILL4** können wir Türen in beliebiger Orientierung im Level plazieren. Die Öffnungs-Geschwindigkeit geben wir in **SKILL6** vor. Über das Vorzeichen der Differenz von **SKILL4** und **SKILL3** ergibt sich, ob die Tür durch Erhöhen oder durch Verringern des Winkels geöffnet wird. **SKILL5** entspricht dem Anfangswinkel der Tür. Die oben definierte Tür **swingdoor** liegt, da der Anfangswinkel 0° beträgt, parallel zur X-Achse.

Prinzipiell funktioniert das Türöffnen mit einer **EACH_TICK**-Aktion wie beim Lift. Nur verändern wir diesmal nicht den Parameter **FLOOR_HGT** einer Region, sondern per **ROTATE**-Befehl ihren Winkel.

```
SOUND knarz,<knarz.wav>;    // sehr kurzer Sound (<0.3 sec!)
SKILL door_speed { }        // Zwischenwerte
SKILL angle      { }
```

```
ACTION swing_open {                                // Tür öffnen
    SET      THERE.SKILL7,THERE.SKILL4; // Endwinkel setzen
    IF (THERE.SKILL4<THERE.SKILL3) // Winkel erhöhen oder verringern?
    { SET      THERE.EACH_TICK,swing_dectick; }
    ELSE
    { SET      THERE.EACH_TICK,swing_inctick; }
}

ACTION swing_close {                                // Tür schliessen
    SET      THERE.SKILL7,THERE.SKILL3; // Endwinkel setzen
    IF (THERE.SKILL4<THERE.SKILL3) // Winkel erhöhen oder verringern?
    { SET      THERE.EACH_TICK,swing_inctick; }
    ELSE
    { SET      THERE.EACH_TICK,swing_dectick; }
}

ACTION swing_inctick {
    RULE      door_speed = TIME_CORR*THERE.SKILL6;
    RULE      THERE.SKILL5 += door_speed; // aktuellen Winkel erhöhen
    ROTATE     THERE,door_speed;
    PLAY_SOUND  knarz,0.3,THERE.GENIUS;
    SET        RENDER_MODE,1;
    IF (THERE.SKILL5>=THERE.SKILL7) // Ziel-Winkel erreicht?
    { SET      THERE.EACH_TICK,NULL; }
}

ACTION swing_dectick {
    RULE      door_speed = -TIME_CORR*THERE.SKILL6;
    RULE      THERE.SKILL5 += door_speed; // aktuellen Winkel erhöhen
    ROTATE     THERE,door_speed;
    PLAY_SOUND  knarz,0.3,THERE.GENIUS;
    SET        RENDER_MODE,1;
    IF (THERE.SKILL5<=THERE.SKILL7) // Ziel-Winkel erreicht?
    { SET      THERE.EACH_TICK,NULL; }
}
```

Die Aktion **swing_open** öffnet die Tür, **swing_close** schließt sie, ganz wie beim Lift. Da sich aber die Tür je nach Position entweder durch Verringern oder durch Erhöhen des Winkels öffnen kann, benötigen wir ein paar Vergleiche mehr, um die jeweils richtige Aktion als **EACH_TICK**-Event zu aktivieren. Der Parameter **SKILL7** der Tür-Region wird als 'Zwischenspeicher' für den Ziel-Winkel, der entweder in **SKILL3** oder in **SKILL4** steht, benutzt.

Das eigentliche Bewegen der Tür erfolgt in den Aktionen **swing_inctick** oder **swing_dectick**. Diese verändern ähnlich wie beim Lift den aktuellen Türwinkel und setzen ihn fest auf den Endwert, falls er diesen überschritten hat.

Unterwasser-Regionen

Im Demo-Level können wir nicht nur in den Wasserlöchern herumschwimmen, sondern sogar untertauchen. Wie können wir einen solchen Effekt erreichen?

Eine Wasser-Region ist ein Sonderfall zweier 'übereinandergestapelter' Regionen, von denen die untere unter und die ober über Wasser liegt:

```
REGION under_water {
    FLOOR_HGT -5;
    CEIL_HGT  1;
    FLOOR_TEX mud_tex;
    CEIL_TEX  underwater_tex;
    IF_ARISE  regio_arise;
}
REGION water {
    FLOOR_HGT 1;      // same as under_water's CEIL_HGT!
    FLOOR_TEX water_tex;
    CEIL_HGT  20;
    CEIL_TEX  sky_tex;
    BELOW     under_water;
    IF_DIVE    regio_dive;
}
```

Sobald der Player die Wasser-Region betritt, sinkt er bis zum Boden der unteren Region ein. Hat der Regionsboden eine **IF_DIVE**-Aktion, so ist der 'durchlässig', und der Player wird sich unmittelbar unter Wasser wiederfinden. Einige Spezialeffekte in der **IF_DIVE**-Aktion - Blubber-Sound, Wechsel der Farbpalette, anderes Bewegungsverhalten - sorgen dann für das richtige Unterwasser-Feeling.

```
PALETTE blue_pal { PALFILE <blue.bbm>; }
SKILL underwater { VAL 0; }
```

```
ACTION regio_dive {
    SET      RENDER_MODE,1;
```

```

IF (underwater)      // Ist der Player schon unter Wasser?
{ END; }             // dann Schluß
SET      underwater,1;      // Ab jetzt unter Wasser!
FADE_PAL blue_pal,0.7;      // Unterwasser-Palette setzen
RULE     PLAYER_ARC += 0.3;  // Blickwinkel verändern
RULE     my_size -= 0.3;     // Hysterese
RULE     PLAYER_SIZE -= 0.3;
RULE     PLAYER_Z -= 0.3;
CALL     set_diving;        // Tauch-Modus - s.u.
}

ACTION regio_arise {
  SET      RENDER_MODE,1;
  IF (underwater==0)      // Player schon über Wasser?
  { END; }               // dann Schluß
  SET      underwater,0;
  FADE_PAL blue_pal,0;     // Unterwasser-Palette abschalten
  RULE     PLAYER_ARC -= 0.3; // Blickwinkel normalisieren
  RULE     my_size += 0.3;    // Hysterese
  RULE     PLAYER_SIZE += 0.3;
  RULE     PLAYER_Z += 0.3;
  CALL     set_swimming;     // Schwimm-Modus - s.u.
}

```

Wir wollen unter Wasser mehrere Effekte auslösen: Alle Farben sollen in milchigen Blautönen erscheinen, alles in der Entfernung soll im Blau verschwimmen (Nebel-Effekt) und der Blickwinkel soll einen leichten 'Froschaugen'-Effekt erhalten. Außerdem wollen wir gegebenenfalls das Bewegungsverhalten des Players umschalten. Für die ersten beiden Effekte haben wir uns eine spezielle Unterwasser-Palette **blue_pal** definiert, die lediglich die Farbe Blau enthält.

In der **regio_dive**-Aktion wird anhand des Skills **underwater** überprüft, ob sich der Player vielleicht schon in der Tiefe befindet. Wenn nicht, wird mit der **FADE_PAL**-Anweisung die Unterwasser-Palette **blue_pal** der normalen Palette zu 70% übergeblendet und zugleich der Blickwinkel verändert.

Es folgen drei Zeilen mit Veränderungen der Player-Größe und vertikalen Position, die mit 'Hysterese' kommentiert sind. Wir hatten schon zu Beginn dieses Kapitels bei der Portal-Aktion mit diesem Begriff zu tun. Eine Hysterese ist ein Programmiertrick, um ein wiederholtes 'Hin- und Herschnappen' von Aktionen zu verhindern. In diesem Falle wäre das ein ständiges Ein- und Wiederauftauchen

des Players, das wir durch Tiefersetzen seiner Augenhöhe beim Eintauchen verhindern.

Die **regio_arise**-Aktion ist für das Auftauchen des Players zuständig. Sie macht sämtliche Unterwasser-Veränderungen wieder rückgängig. Wir können sie auslösen, indem wir den Player von unten mit dem Kopf die Wasserfläche berühren lassen. Das ginge am besten mit Hilfe einer Tauch-Bewegungsaktion, die wir hier jedoch dem Leser zur Übung überlassen wollen. Die MOVE.WDL der VRDEMO enthält Bewegungs-Aktionen für Tauchen und Schwimmen.

Donnerstag: Künstliche Intelligenz

Wie wir mittlerweile erfahren haben, bereitet *ein* Bereich der Spielentwicklung immer wieder großes Kopfzerbrechen: Wie definiere ich ein möglichst intelligentes Verhalten der Gegner? Bevor wir dieses Tutorial endgültig beenden, wollen wir uns darum noch ein wenig mit der Schaffung von intelligentem künstlichem Leben befassen.

Um unsere virtuelle Welt mit Lebewesen zu bevölkern, benutzen wir die **ACTOR**-Definition. *Actors* nennen wir diejenigen Objekte, die sich selbständig bewegen können. Wie wir der WDL-Referenz entnehmen können, werden sie ähnlich wie Things definiert. Es gibt nur ein paar zusätzliche Flags und Schlüsselworte: **TARGET**, **WAYPOINT**, **SPEED** und **VSPEED**.

Mit dem Schlüsselwort **TARGET** können wir dem Actor ein primitives Bewegungsverhalten geben: Er kann einen vorgegebenen Weg abschreiten oder sich auf ein Ziel zu- bzw. davon wegbewegen. Würden wir uns im Game jedoch darauf beschränken, so würde das Ganze rasch langweilig. Monster, die als Gegner ernstgenommen werden wollen - und welches Monster will das nicht? - sollten sich 'realistisch' verhalten. Sie müssen intelligent auf den Player reagieren, ihm ausweichen, solange er stark ist, und Jagd auf ihn machen, wenn er Schwächen zeigt. Dazu ist eine Art künstlicher Intelligenz erforderlich.

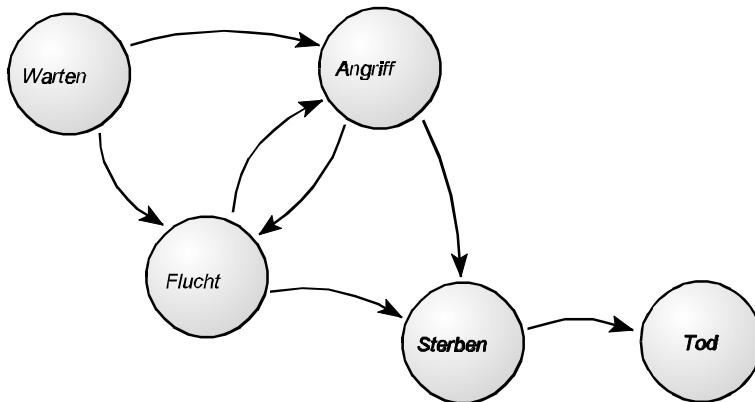
In diesem Tutorial-Kapitel lernen Sie, wie Sie Ihren Kreaturen eine elektronische Persönlichkeit verpassen können. Wir machen dazu einen Abstecher in die Theorie der linearen Automaten. Am Schluß des Kapitels werden Sie in der Lage sein, elektronische Lebewesen zu erschaffen, die in der Komplexität ihres Verhaltens einfachen Organismen gleichen.

Lineare Automaten

Unter dem Begriff 'Künstliche Intelligenz' versteht man Methoden, die einer Maschine die Fähigkeit verleihen, sinnvolle Entscheidungen zu treffen. Dabei ist es für unsere Zwecke unerheblich, ob die Maschine tatsächlich Intelligenz besitzt. Sie soll sich nur möglichst intelligent *verhalten*. Ob sich aus dem Verhalten einer Maschine tatsächlich Rückschlüsse auf deren Intelligenz ziehen lassen, wird von Behaviouristen, Mechanisten, Dualisten und Anhängern anderer theoretischer Schulen zwar seit vielen Jahren diskutiert. Diese zweifellos interessante Frage hat aber auf die WDL-Programmierung nicht den geringsten Einfluß.

Ist das Verhalten unserer Maschine von gewisser Einfachheit, so läßt sich diese als *linearer Automat* beschreiben. Das Verhalten eines linearen Automaten können wir als eine Anzahl säuberlich getrennter einzelner Verhaltensmuster betrachten. Jedes dieser Verhaltensmuster entspricht einem *inneren Zustand*. Ein linearer Automat verfügt also über eine endliche Anzahl von Zuständen; er 'befindet' sich immer in einem dieser Zustände. Die Zustände stellen gewissermaßen das 'Innenleben' des schwarzen Kastens dar. Für jeden Zustand gibt es Bedingungen, beispielsweise äußere Reize, unter denen der Automat in einen anderen Zustand übergeht.

Bei unserem Actor-Automaten könnte man zum Beispiel folgende Verhaltensmuster - Zustände - definieren: Warten, Angriff, Flucht, Sterben und Tod.



Die Pfeile deuten die *Übergänge* zwischen den Zuständen an. Solche Übergänge werden in der WDL durch *Events* ausgelöst. In der folgenden Liste sind einige Events aufgeführt, auf die ein Actor mit Zustandsübergängen reagieren kann:

Event	Realisiert durch
# Actor erblickt den Player.	EACH_TICK -Aktion mit periodischer Auswertung einer SHOOT -Anweisung.
# Der Player kommt in die Nähe des Actors (oder umgekehrt).	IF_NEAR -Aktion mit DIST > 0.
# Player und Actor berühren sich.	IF_NEAR -Aktion mit DIST = 0.
# Der Actor hat sich über eine bestimmte Distanz hinaus vom Player entfernt.	IF_FAR -Aktion.
# Der Actor befindet sich in der Nähe eines Objekts, welches eine EXPLODE -Anweisung auslöst.	IF_HIT -Aktion mit gesetztem FRAGILE -Flag.
# Der Player trifft den Actor mit der SHOOT -Anweisung.	IF_HIT -Aktion.
# Ein Animationszyklus der Actor-Textur ist abgelaufen.	EACH_CYCLE -Aktion.
# Ein Skill, der von einer anderen Aktion ständig verändert wird, hat einen bestimmten Wert erreicht.	EACH_TICK -Aktion, in der dieser Skill ständig mit seinem Grenzwert verglichen wird.
# Eine bestimmte Zeitspanne ist verstrichen.	EACH_TICK -Aktion mit 'Herunterzählen' eines Skills, der als Zeitmesser dient.

Event	Realisiert durch
# Der Actor hat einen bestimmten Waypoint erreicht.	IF_ARRIVED -Aktion bei Verfolgen eines WAY , gegebenenfalls mit Vergleich der Waypoint-Nummer.
# Der Actor hat eine bestimmte Region erreicht.	IF_ARRIVED -Aktion ohne Way, gegebenenfalls mit Vergleich eines Region-Skills.
# Der Actor ist mit einem Hindernis kollidiert.	IF_HIT -Aktion mit TARGET BULLET .
# Zufallsereignis.	EACH_TICK -Aktion, die periodisch den Skill RANDOM mit einer festen Zahl vergleicht.

All diese Events lassen sich natürlich auch miteinander kombinieren. Für jeden Zustands-Übergang ist eine WDL-Aktion erforderlich, die mit einer Anzahl von SET-Anweisungen den neuen Zustand herstellt.

Außer Zuständen können wir für den Automaten noch innere Variablen definieren, die verschiedene Werte annehmen können und ihrerseits das Verhalten beeinflussen. Unsere Actor-Skills entsprechen solchen Variablen. Ihr Wert kann in einer Aktion ausgewertet werden und z.B. über den Übergang von einem in einen anderen Zustand entscheiden. Zwar kennt die 'reine' Automatentheorie keine Skills, doch könnte man theoretisch jeden Actor-Skill durch eine endliche Anzahl von 'Unter'-Zuständen ersetzen, die bestimmten Werten dieses Skills entsprechen.

Die Zustandstabelle

Die Aufgliederung des Verhaltens unseres Automaten in Zustände und Übergänge erlaubt es uns, die Automaten-WDL einigermaßen übersichtlich zu halten. Um das Ganze zu verdeutlichen, wollen wir nun unseren ersten Automaten definieren. Wir benutzen dazu den Tron-Virus aus dem Computerspiel SKAPHANDER (ein paar Texturen dazu zeichnen Sie sich am besten selbst). Bevor wir die erste Zeile

WDL-Code schreiben, machen wir uns ein theoretisches Konzept. Welche Zustände soll unser Tron erhalten, wie sollen sich die Zustände unterscheiden, welche Skills und welche Übergänge brauchen wir?

Wir legen fest: Bei Spielstart soll sich der Tron im Zustand WARTEN befinden. In diesem Zustand hockt er in irgendeiner dunklen Ecke und lauert auf den Player. Kommt der Player in die Nähe des Tron, so geht dieser in den Zustand ANGRIFF über. Trifft der Player den Tron mit der SHOOT-Anweisung, so richtet sich der Übergang nach der Zahl der Treffer, die der Tron bereits hat einstecken müssen: Der neue Zustand ist entweder ANGRIFF oder FLUCHT oder STERBEN.

Um die Anzahl der Treffer zu zählen, verwenden wir den SKILL1 des Tron. Zur besseren Übersichtlichkeit fassen wir den Zustand in einer *Zustandstabelle* zusammen.

Zustand	Textur	Target	Player nahe	Treffer, bisher ≤ 4	Treffer, bisher ≤ 5	Treffer, bisher > 5	Anim zu Ende
WARTEN	tron_tex	NULL	ANGRIFF	ANGRIFF	FLUCHT	STERBEN	-

Wir haben für den Tron fünf Events definiert, die seinen Zustand ändern können. Drei davon werden durch Treffer ausgelöst, jeweils abhängig von der Zahl der bisherigen Treffer. Außerdem sind noch die jeweilige Textur und das TARGET pro Zustand angegeben. Wir können nun die Zustandstabelle um die restlichen Zustände ergänzen. Im Zustand FLUCHT soll dabei ein weiterer Treffer den Tron wieder zum Angriff animieren:

Zustand	Textur	Target	Player nahe	Treffer, bisher ≤ 4	Treffer, bisher ≤ 5	Treffer, bisher > 5	Anim zu Ende
WARTEN	tron_tex	NULL	ANGRIFF	ANGRIFF	FLUCHT	STERBEN	-
ANGRIFF	tron_atak	FOLLOW	-	ANGRIFF	FLUCHT	STERBEN	-
FLUCHT	tron_hovo	REPEL	ANGRIFF	ANGRIFF	ANGRIFF	STERBEN	-
STERBEN	tron_explo	NULL	-	-	-	-	TOD
TOD	tron_rest	NULL	-	-	-	-	-

Leere Tabellenfelder (-) ändern den Zustand des Actors nicht. Unser Actor ist ein einfacher Automat; bei komplexerem Verhalten bekäme unsere Zustandstabelle wesentlich mehr Spalten und Reihen. Bei genauem Betrachten der Tabelle fällt vielleicht auf, daß einige überflüssige Übergänge definiert worden sind. So wird z.B. der Übergang von WARTEN zu STERBEN nie erreicht, da der Tron bereits bei dem ersten Treffer zum ANGRIFF übergeht. Da man aber solche Sonderfälle bei komplexeren Automaten nicht mehr so leicht erkennt, kann es nie schaden, die Tabelle so vollständig wie möglich zu definieren.

Ist die Zustandstabelle fertig, geht es an's Programmieren. Zuerst definieren wir die Sounds, Bitmaps und Texturen für die verschiedenen Zustände des Tron sowie den Actor selbst:

```

////////////////////////////////////
// Tron-Actor
////////////////////////////////////
SOUND tron_snd,<tron_vir.wav>;
SOUND tron_baller,<schuss1.wav>;
SOUND tron_aua,<tron_au.wav>;
SOUND tron_spot,<tr_tac.wav>;
////////////////////////////////////
BMAP  tron_0vh1,<v3_vhs.lbm>,20,12,92,84;
BMAP  tron_0vh2,<v3_vhs.lbm>,180,12,92,84;
BMAP  tron_0vh3,<v3_vhs.lbm>,340,12,92,84;
BMAP  tron_0vh4,<v3_vhs.lbm>,20,117,92,84;
BMAP  tron_0vh5,<v3_vhs.lbm>,180,117,92,84;
BMAP  tron_0vh6,<v3_vhs.lbm>,340,117,92,84;

BMAP  tron_0hre1,<v3_re_li.lbm>,20,12,92,84;
BMAP  tron_0hre2,<v3_re_li.lbm>,180,12,92,84;
BMAP  tron_0hre3,<v3_re_li.lbm>,340,12,92,84;
BMAP  tron_0hre4,<v3_re_li.lbm>,20,117,92,84;
BMAP  tron_0hre5,<v3_re_li.lbm>,180,117,92,84;
BMAP  tron_0hre6,<v3_re_li.lbm>,340,117,92,84;

BMAP  tron_0s1,<v3_vhs.lbm>,20,222,92,84;
BMAP  tron_0s2,<v3_vhs.lbm>,180,222,92,84;
BMAP  tron_0s3,<v3_vhs.lbm>,340,222,92,84;
BMAP  tron_0s4,<v3_vhs.lbm>,20,327,92,84;
BMAP  tron_0s5,<v3_vhs.lbm>,180,327,92,84;

```

```

BMAP   tron_0s6,<v3_vhs.lbm>,340,327,92,84;

BMAP   tron_0hli1,<v3_re_li.lbm>,20,222,92,84;
BMAP   tron_0hli2,<v3_re_li.lbm>,180,222,92,84;
BMAP   tron_0hli3,<v3_re_li.lbm>,340,222,92,84;
BMAP   tron_0hli4,<v3_re_li.lbm>,20,327,92,84;
BMAP   tron_0hli5,<v3_re_li.lbm>,180,327,92,84;
BMAP   tron_0hli6,<v3_re_li.lbm>,340,327,92,84;

BMAP   tron_an2,<v3_an.pcx>,180,12,92,84;
BMAP   tron_an3,<v3_an.pcx>,340,12,92,84;
BMAP   tron_an4,<v3_an.pcx>,20,117,92,84;
BMAP   tron_an5,<v3_an.pcx>,180,117,92,84;
BMAP   tron_an6,<v3_an.pcx>,340,117,92,84;

BMAP   tron_ex1,<v3_ex.lbm>,160,0,160,105;
BMAP   tron_ex2,<v3_ex.lbm>,320,0,160,105;
BMAP   tron_ex3,<v3_ex.lbm>,0,105,160,105;
BMAP   tron_ex4,<v3_ex.lbm>,160,105,160,105;
BMAP   tron_ex5,<v3_ex.lbm>,320,105,160,105;
BMAP   tron_ex6,<v3_ex.lbm>,0,210,160,105;
BMAP   tron_ex7,<v3_ex.lbm>,160,210,160,105;
BMAP   tron_ex8,<v3_ex.lbm>,320,210,160,105;
BMAP   tron_ex9,<v3_ex.lbm>,0,315,160,105;
BMAP   tron_ex10,<v3_ex.lbm>,160,315,160,105;
BMAP   tron_ex11,<v3_ex.lbm>,320,315,160,105;

BMAP   tron_corpse,<knallig.lbm>,437,256,47,21;

```

```

////////////////////////////////////

```

```

TEXTURE   tron_tex {
    SCALE_XY   16,16;
    SIDES      8;
    CYCLES     6;
    BMAPS   tron_0vh1,tron_0vh2,tron_0vh3,tron_0vh4,tron_0vh5,tron_0vh6,
            tron_0hre1,tron_0hre2,tron_0hre3,tron_0hre4,tron_0hre5,tron_0hre6,
            tron_0s1,tron_0s2,tron_0s3,tron_0s4,tron_0s5,tron_0s6,
            tron_0hre1,tron_0hre2,tron_0hre3,tron_0hre4,tron_0hre5,tron_0hre6,
            tron_0vh1,tron_0vh2,tron_0vh3,tron_0vh4,tron_0vh5,tron_0vh6,
            tron_0hli1,tron_0hli2,tron_0hli3,tron_0hli4,tron_0hli5,tron_0hli6,
            tron_0s1,tron_0s2,tron_0s3,tron_0s4,tron_0s5,tron_0s6,
            tron_0hli1,tron_0hli2,tron_0hli3,tron_0hli4,tron_0hli5,tron_0hli6;
    DELAY     3,3,2,2,3,3;
}

```

```
MIRROR    0,0,0,1,1,1,1,0;
AMBIENT    0.1;
SOUND      tron_snd;
SCYCLE     1;
SVOL       0.2;
SDIST      40;
}

TEXTURE    tron_hovo {
  SCALE_XY  16,16;
  SIDES      8;
  CYCLES     6;
  BMAPS     tron_0vh1,tron_0vh2,tron_0vh3,tron_0vh4,tron_0vh5,tron_0vh6,
            tron_0hre1,tron_0hre2,tron_0hre3,tron_0hre4,tron_0hre5,tron_0hre6,
            tron_0s1,tron_0s2,tron_0s3,tron_0s4,tron_0s5,tron_0s6,
            tron_0hre1,tron_0hre2,tron_0hre3,tron_0hre4,tron_0hre5,tron_0hre6,
            tron_0vh1,tron_0vh2,tron_0vh3,tron_0vh4,tron_0vh5,tron_0vh6,
            tron_0hli1,tron_0hli2,tron_0hli3,tron_0hli4,tron_0hli5,tron_0hli6,
            tron_0s1,tron_0s2,tron_0s3,tron_0s4,tron_0s5,tron_0s6,
            tron_0hli1,tron_0hli2,tron_0hli3,tron_0hli4,tron_0hli5,tron_0hli6;
  DELAY      5,5,3,3,5,5;
  RANDOM     0.3;
  MIRROR     0,0,0,1,1,1,1,0;
  AMBIENT    -0.2;
  SOUND      tron_snd;
  SCYCLE     1;
  SVOL       0.2;
  SDIST      40;
}

TEXTURE    tron_atak {
  SCALE_XY  16,16;
  CYCLES     7;
  BMAPS     tron_0vh1,tron_an2,tron_an3,tron_an4,tron_an5,tron_an6,tron_an3;
  DELAY      1,1,1,1,1,1,1;
  AMBIENT    0.3;
  SOUND      tron_baller;
  SCYCLE     4;
  SVOL       0.25;
  SDIST      200;
}

TEXTURE    tron_explo {
```

```

SCALE_XY  18,18;
CYCLES    11;
BMAPS     tron_ex1,tron_ex2,tron_ex3,tron_ex4,tron_ex5,tron_ex6,
          tron_ex7,tron_ex8,tron_ex9,tron_ex10,tron_ex11;
DELAY     1,2,2,2,2,2,2,3,3,2,2;
AMBIENT    1;
FLAGS     ONESHOT;
SOUND     tron_bang;
SCYCLE    2;
SVOL      0.5;
SDIST     300;
}

TEXTURE   tron_rest {
    SCALE_XY  16,16;
    BMAPS     tron_corpse;
}

////////////////////////////////////
ACTOR   simple_tron {
    TEXTURE   tron_tex;
    SKILL1    0;           // Anfangs-Trefferzahl
    HEIGHT    2;           // Schweben-Höhe
    EACH_TICK tron_warten; // Anfangszustand
}

```

Die Texturen sind teilweise mehrseitig, wobei wegen der Symmetrie des Actors **MIRROR**-Flags eingesetzt werden können, um Bitmaps zu sparen. Da der Tron beim Angriff stets auf den Player zurennt, genügt hier eine einseitige Textur; das gleiche gilt für die Explosion des Tron. Die Flucht-Textur wurde mit einem negativen Ambient etwas abgedunkelt, um eine Beschädigung oder einen Energieverlust anzudeuten.

Die Definition des Actors kann sehr sparsam bleiben, da alle nötigen Parameter in der Aktion **tron_warten** gesetzt werden. Diese Aktion ist als **EACH_TICK**-Aktion definiert. Mit diesem Trick erreichen wir, daß unmittelbar nach Spielbeginn der Zustand WARTEN aktiviert wird. Wir müssen allerdings dafür sorgen, daß die Aktion selbst den **EACH_TICK**-Event wieder auf **NULL** setzt, sonst würde sie ständig wiederholt:

```
////////////////////////////////////////
ACTION tron_warten {
    SET    MY.TEXTURE,tron_tex;
    SET    MY.TARGET,NULL;
    SET    MY.DIST,60;           // Reaktionsradius
    SET    MY.IF_NEAR,tron_angriff;
    SET    MY.IF_HIT,tron_anfluster;
    SET    MY.EACH_TICK,NULL;
    SET    MY.EACH_CYCLE,NULL;
}

ACTION tron_anfluster {
    RULE    MY.SKILL1 += 1;       // Treffer-Zähler erhöhen
    PLAY_SOUND    tron_aua;      // Wehgeschrei
    IF (MY.SKILL1>5)
    { CALL    tron_sterben; }
    IF (MY.SKILL1==5)
    { CALL    tron_flucht; }
    CALL    tron_angriff;
}
}
```

Da alle Aktionen vom Actor selbst ausgelöst werden, können wir zur Bezeichnung des Actors immer das MY-Synonym verwenden. Die Aktion **tron_anfluster** ist erforderlich, um je nach Anzahl der vorherigen Treffer einen der Zustände ANGRIFF, FLUCHT oder STERBEN zu aktivieren. Außerdem soll sie als Reaktion auf Treffer einen Sound abspielen.

```
ACTION tron_angriff {
    SET    MY.TEXTURE,tron_atak;
    SET    MY.SPEED,0.4;
    SET    MY.VSPEED,0.2;
    SET    MY.CAREFULLY,1;       // Hindernissen ausweichen
    SET    MY.TARGET,FOLLOW;
    SET    MY.IF_NEAR,NULL;
    SET    MY.IF_HIT,tron_anfluster;
    SET    MY.EACH_TICK,NULL;
    SET    MY.EACH_CYCLE,tron_schuss;
}

ACTION tron_schuss {
    SET    SHOOT_X,0;
}
```

```
SET      SHOOT_Y, -0.5;          // Tron schießt mit den Beinen
SET      SHOOT_RANGE, 200;
SHOOT    MY;
IF (HIT_DIST>0)                  // Treffer?
{ RULE   health -= 1; }          // Player steckt Treffer ein
}
```

Bei der Angriffs-Aktion müssen wir zugleich mit dem Target auch die horizontale und vertikale Geschwindigkeit sowie gegebenenfalls das **CAREFULLY**-Flag setzen. Eine vertikale Geschwindigkeit **VSPEED** in Kombination mit Target **FOLLOW** bewirkt, daß der Tron auf den Player zuschwebt, wenn dieser sich z.B. auf einem Sims über ihm befindet.

Die Angriffs-Textur ist so gezeichnet, daß der Tron pro Animationszyklus einen Schuß abgibt. Daher brauchen wir die **EACH_CYCLE**-Aktion `tron_schuss`, um festzustellen, ob der Player tatsächlich getroffen wurde. Der **EACH_CYCLE**-Event löst hier also keinen neuen Zustand aus!

Die **SHOOT**-Anweisung prüft, ob sich vom Zentrum des Actors bis zum Player eine waagerechte Linie ziehen läßt, die keine Wände schneidet. Ist das der Fall - und befindet sich der Player innerhalb der Entfernung **SHOOT_RANGE** - so wird der vordefinierte Skill **HIT_DIST** entsprechend der Entfernung des Actors gesetzt. Andernfalls ist **HIT_DIST** 0.

Mit **SHOOT_Y** verschieben wir das Schußzentrum leicht nach unten, da der Tron mit den 'Beinen' schießt. Würden wir das nicht tun, so würde der Player auch dann getroffen, wenn die Energie-Geschütze des Tron durch eine Mauer verdeckt wären. Damit das Spiel realistisch wirkt, sollten wir immer auf solche Feinheiten achten.

```
ACTION tron_flucht {
SET    MY.TEXTURE, tron_hovo;
SET    MY.SPEED, 0.4;
SET    MY.VSPEED, 0;
SET    MY.CAREFULLY, 1;          // Hindernissen ausweichen
SET    MY.TARGET, REPEL;
SET    MY.IF_NEAR, NULL;
SET    MY.IF_HIT, tron_angster;
```



```
SET    MY.EACH_TICK, NULL;
SET    MY.EACH_CYCLE, NULL;
}

ACTION tron_angster {
    RULE    MY.SKILL1 += 1;          // Treffer-Zähler erhöhen
    PLAY_SOUND    tron_aua;        // Wehgeschrei
    IF (MY.SKILL1>5)
    { CALL    tron_sterben; }
    ELSE
    { CALL    tron_angriff; }
}

ACTION tron_sterben {
    SET    MY.TEXTURE, tron_explo;
    SET    MY.SPEED, 0.2;
    SET    MY.VSPEED, 0;
    SET    MY.CAREFULLY, 0;
    SET    MY.PASSABLE, 1;
    SET    MY.TARGET, FOLLOW;
    SET    MY.IF_NEAR, NULL;
    SET    MY.IF_HIT, NULL;
    SET    MY.EACH_TICK, NULL;
    SET    MY.EACH_CYCLE, tron_tod;
}

ACTION tron_tod {
    SET    MY.TEXTURE, tron_rest;
    SET    MY.PASSABLE, 1;
    SET    MY.TARGET, NULL;
    SET    MY.IF_NEAR, NULL;
    SET    MY.IF_HIT, NULL;
    SET    MY.EACH_TICK, NULL;
    SET    MY.EACH_CYCLE, NULL;
}
```

Die Aktion **tron_sterben** enthält wieder eine Merkwürdigkeit, die anscheinend im Widerspruch zu unserer Zustandstabelle steht. Obwohl der Tron schon erledigt ist, bewegt er sich wieder mit Target **FOLLOW** auf den Player zu! Durch diesen Trick erreichen wir, daß sich die Explosionswolke zusätzlich zu ihrer eigenen Animation nähert und somit auseinanderbewegt, was die Explosion noch realistischer aussehen läßt. Natürlich darf die Wolke keinen Hindernissen mehr

ausweichen, deshalb ist **CAREFULLY** auf 0 gesetzt. Außerdem muß beim explodierenden und auch beim toten Tron **PASSABLE** gesetzt sein, damit er für den Player kein Hindernis darstellt.

Fortgeschrittene Automaten

Intelligentes und 'persönliches' Verhalten von Gegnern trägt besonders bei Rollen- oder Actionspielen wesentlich zum Spielspaß bei. Wenn Sie ein solches Spiel eine Zeitlang spielen, werden Ihnen an Ihren elektronischen Gegnern vielleicht bestimmte Merkmale - Schwächen oder Eigenheiten - auffallen. Diese Schwächen können Sie sich zunutze machen. Sie können eine *Strategie* entwickeln. Fehlen diese Merkmale, so entfällt auch jede Strategie: Das Spiel ist ein bloßes Ballerspiel. Solche Spiele werden schnell langweilig.

Was kann man an unserem Actor **simple_tron** verbessern? Am naheliegendsten sind die Möglichkeiten *kosmetischer* Verbesserungen. Einfache Zustands-Erweiterungen geben dem Actor ein komplexeres 'persönliches' Verhaltens-Repertoire. Wenn er den Player erblickt, stößt er einen Wutschrei aus; bei Treffern prallt er zurück oder wird in Drehung versetzt. Er bewegt sich nicht nur geradlinig auf den Player zu, sondern weicht in Zufallsabständen seitlich auch, um nicht selbst getroffen zu werden. Einige dieser Möglichkeiten sind in der mitgelieferten VIREN.WDL realisiert; die Anzahl der Zustände und Übergänge erhöht sich dadurch allerdings um einiges.

Wesentlich komplizierter sind Verbesserungen im *strategischen* Verhalten des Actors. Diese gehen deutlich in Richtung einer künstlichen Intelligenz und lassen sich nur dadurch realisieren, daß Actors miteinander kommunizieren (über **EXPLODE**-Anweisungen und Player-Skills) und/oder daß sie in ihrem Verhalten die Position des Players relativ zur Topographie des Levels berücksichtigen. Auch hier wieder einige Vorschläge:

Sobald ein Actor (Wächter) den Player erblickt, alarmiert er andere Actors innerhalb einer bestimmten Entfernung. Zum Alarm-Auslösen eignen sich auch Objekte wie Sensoren, Kameras, Stolperdrähte oder Lichtschranken. Wichtig für die Spannung ist, daß auch der Player den Alarm mitbekommt!

Wenn mehrere Actors den Player angreifen, versuchen sie ihn zu umzingeln und einzukreisen (Indianertaktik).

Actors lauern dem Player hinter Ecken auf, indem sie sich ihm nur gerade soweit nähern, daß sie noch nicht gesehen werden können.

Es gibt Schluchten oder Räume im Level, die als 'Fallen' fungieren: Actors können den Player hier von allen Seiten attackieren. Die Actors versuchen, den Player in solche Fallen zu locken oder abzudrängen.

Actors können den Kampfausgang prognostizieren, indem sie die Stärke des Players mit ihrer eigenen Stärke vergleichen. Wenn er noch viel Kraft hat, rufen sie Verstärkung; erst wenn er schwach ist, greifen sie direkt an.

Einige dieser Vorschläge sind in manchen Leveln des kommerziellen **SKAPHANDER**-Spiels realisiert. Wir dürfen aber des Guten nicht zuviel tun: Der Player soll immer mitbekommen, daß sich Unheil anbahnt! Actors, die den Player lange Zeit unbemerkt belauern, um ihn dann urplötzlich von allen Seiten fertigzumachen, tragen nicht unbedingt zum Spielspaß bei.

Spezial-Automaten

Um Games aufzuwerten, können wir in den Levels Objekte plazieren, die ebenfalls auf den Spieler bzw. auf Treffer oder Betätigung auf spezielle Weise reagieren. Als Beispiel für die Realisierung von 'Special Effects' in WDL nehmen wir wieder zwei Objekte aus dem SKAPHANDER-Spiel, nämlich das Fass und die Mine.

Das Fass soll bei jedem Treffer hochhüpfen; beim Aufschlag auf dem Boden soll eine Texturanimation ablaufen. Durch das Hüpf-Verhalten kann der Player - gewissermaßen als Geschicklichkeitsübung - Fässer in der Luft tanzen lassen.

```
BMAP fass1,<tonne.lbm>,23,254,81,174;
```

```
BMAP fass2,<tonne.lbm>,117,254,81,174;
```

```
BMAP fass3,<tonne.lbm>,236,254,81,174;
```

```
BMAP fass4,<tonne.lbm>,340,254,81,174;
```

```
BMAP fass5,<tonne.lbm>,440,254,81,174;
```

```
TEXTURE fass_tex {
    SCALE_XY  35,35;
    CYCLES     5;
    BMAPS      fass1,fass2,fass3,fass4,fass5;
    FLAGS      ONESHOT;
    DELAY       3,3,3,3,3;
}

THING fass {
    TEXTURE     fass_tex;
    HEIGHT      0;
    FLAGS       FRAGILE;           // Fass reagiert auch auf EXPLODE
    IF_HIT      fass_hit;
}

SKILL hupf_abweich  { VAL 0; }

ACTION fass_hit {
    IF (MY.RESULT<0.1)           // Kein echter Schuss?
    { END; }
    PLAY_SOUND  banngg,0.2;
    RULE        MY.SKILL1 = 1+hupf_abweich; // Anfangsgeschwindigkeit
    RULE        hupf_abweich += 0.05;      // damit Faesser verschieden hüpfen
    IF (hupf_abweich>0.25)
    { RULE hupf_abweich = -0.15; }
    SET         MY.EACH_TICK,fass_hupf;
}
```

In der Action **fass_hit** wird zuerst der Parameter **RESULT** geprüft. Nur bei einem **RESULT**-Wert > 0 handelt es sich um einen 'echten' Schuss mit einer Waffe, d.h. um eine **SHOOT**-Anweisung mit **SHOOT_FAC** > 0. In unserem Demo-Level wird nämlich **SHOOT** auch z.B. zum Öffnen von Türen benutzt und löst ebenfalls eine **IF_HIT**-Aktion aus, die dann natürlich nicht zum Hüpfen führen darf.

Zu Beginn wird der **SKILL1** entsprechend der Geschwindigkeit gesetzt, mit der sich das Fass aufwärts bewegen soll. Zu dieser Anfangsgeschwindigkeit addieren wir den Skill **hupf_abweich**, den wir daraufhin leicht verändern. Damit erreichen wir, daß jedes Fass mit einer geringfügig anderen Geschwindigkeit loshüpft; ganze Faßreihen, die durch eine **EXPLODE**-Anweisung zum Hüpfen gebracht werden,

sehen dadurch besser aus. (Alternativ könnten wir z.B. die Anfangsgeschwindigkeit vom **RESULT**-Parameter abhängig machen).

Schließlich wird eine **EACH_TICK**-Aktion gestartet, die für die weitere Aufwärts-Bewegung sorgt:

```
SKILL decken_distanz    { VAL 0; }

ACTION fass_hupf {
    SET      RENDER_MODE,1;    // damit Fassbewegung sichtbar ist
    // Abstand von Decke berechnen
    RULE    decken_distanz = THERE.CEIL_HGT - THERE.FLOOR_HGT - MY.SIZE_Y;
    RULE    MY.HEIGHT += MY.SKILL1;    // Fass vertikal bewegen
    IF (MY.HEIGHT >= decken_distanz)    // beruehrt es die Decke?
    { RULE    MY.HEIGHT = decken_distanz; } // nicht weiter steigen

    RULE    MY.SKILL1 += -0.15*TIME_CORR;    // Gravitation
    IF (MY.HEIGHT > 0)                // Fass noch in der Luft?
    { END; }

    RULE    MY.HEIGHT = 0;
    PLAY_SOUND    rrumms,0.3;    // Aufschlag
    SET      MY.EACH_TICK,NULL;
    SET      MY.PLAY,1;    // Texturanimation starten
}
```

RENDER_MODE muss bekanntlich immer auf 1 gesetzt werden, wenn sich an den Positionen von Things, Walls oder Regionen irgend etwas verändert. Dann wird die Fasshöhe bei Deckenberührung berechnet. Zur augenblicklichen **HEIGHT** des Objekts wird die Vertikalgeschwindigkeit (**SKILL1**) addiert und schließlich getestet, ob die Decke berührt wurde; wenn ja, wird das Fass genau unter die Decke gesetzt.

Jetzt kommt wieder die Schwerkraft ins Spiel, die bewirkt, daß sich die Vertikalgeschwindigkeit eines Objekts pro Zeiteinheit um einen festen Betrag verringert. Wir müssen hier den **TIME_CORR**-Skill verwenden, um unterschiedliche Rechnergeschwindigkeiten auszugleichen. Das ständige Verringern der Vertikalgeschwindigkeit bewirkt, daß diese irgendwann negativ wird, so daß das Fass wieder dem Erdboden zufällt. Ist der Boden erreicht, wird ein Sound und eine

Animation abgespielt und die **EACH_TICK**-Aktion abgeschaltet. Da **HEIGHT** durch das ständige Verringern auch negativ geworden sein kann, muß dieser Parameter explizit auf 0 gesetzt werden.

Wir haben mit dem Fass einen sehr einfachen Automaten mit nur zwei Zuständen - auf dem Boden und in der Luft - kennengelernt. Komplizierter wird die Sache bei der Mine, die aufgenommen, getragen, geworfen und zur Explosion gebracht werden kann:

```

////////////////////////////////////
// MINE ZUM ABLEGEN UND ABHAUEN
////////////////////////////////////
SOUND klick,<ausklink.wav>;
SOUND klonk,<aufsetz.wav>;
SOUND tic_tac,<tic.wav>;
////////////////////////////////////
BMAP min_ob,<mine2.lbm>,0,0,78,80;
BMAP min_ex1,<knallig.lbm>,0,42,128,128;
BMAP min_ex2,<knallig.lbm>,128,42,128,128;
BMAP min_ex3,<knallig.lbm>,256,42,128,128;
BMAP min_ex4,<knallig.lbm>,384,42,128,128;
BMAP min_ex5,<knallig.lbm>,0,170,128,128;
BMAP min_ex6,<knallig.lbm>,128,170,128,128;

TEXTURE  mine_tex {
    SCALE_XY  32,32;
    BMAPS      min_ob;
}

TEXTURE  mine_tick_tex {
    SCALE_XY  32,32;
    CYCLES    2;
    BMAPS      min_ob,min_ob;
    DELAY      4,4;
    SOUND      tic_tac;
    SCYCLE     1;
    SVOL       0.4;
    SDIST      60;
}

TEXTURE  mine_explo_tex {

```

```

SCALE_XY  10,10;
CYCLES     6;
BMAPS      min_ex1,min_ex2,min_ex3,min_ex4,min_ex5,min_ex6;
DELAY      2,1,1,1,3,2;
SCYCLES     1,0,1,0,0,0;
AMBIENT     1;
FLAGS      ONESHOT;
SOUND       tron_bang;
SVOL        0.6;
SDIST       350;
}

```

```
OVLY min_ovl,<mine2.lbm>,119,0,40,31;
```

```

OVERLAY mine_ovr {
    POS_X      138;
    POS_Y      293;
    OVLYS      min_ovl;
}

```

```

////////////////////////////////////
ACTOR mine {
    TEXTURE     mine_tex;
    FLAGS       FRAGILE;
    IF_NEAR     nimm_mine;
}

```

Es fällt auf, daß bei der animierten Textur mine_tick_tex zwischen zwei gleichen Bitmaps gewechselt wird. Die Animation hat hier nur einen akustischen Zweck, sie soll ein rhythmisches Ticken erzeugen.

```

ACTION nimm_mine {
    PLAY_SOUND  beamer,0.3;
    SET         MY.INVISIBLE,1;
    SET         mine1_ovr.VISIBLE,1;
    SET         IF_CTRL,mine_werf;
    SET         IF_LEFT,mine_werf;
}

```

Ein klassischer Zustandsübergang: Beim Berühren der Mine verschwindet diese aus der Landschaft und erscheint dafür als Overlay auf dem Bildschirm (mehr über

Overlays erfahren Sie im nachfolgenden Kapitel). Mit der linken Maustaste oder der **[Strg]**-Taste kann man nun den Minen-Wurf auslösen:

```
ACTION mine_werf {
    SET      IF_CTRL,NULL;           // Tasten deaktivieren
    SET      IF_LEFT,NULL;
    SET      mine1_ovr.VISIBLE,0;    // Overlay verschwinden lassen
    PLAY_SOUND klick,0.5;

    SET      mine.HEIGHT,1.5;
    SET      mine.DIST,0;
    DROP      mine;                  // An Player-Position platzieren

    SET      mine.TEXTURE,mine_tex;
    SET      mine.INVISIBLE,0;       // Objekt jetzt wieder sichtbar

    SET      mine.ANGLE,PLAYER_ANGLE;
    SET      mine.SPEED,2;
    SET      mine.VSPEED,0.4;        // Vertikalgeschwindigkeit
    SET      mine.CAREFULLY,1;
    SET      mine.TARGET,MOVE;       // In Bewegung setzen

    SET      mine.EACH_TICK,wurf_flug;
    SET      mine.EACH_CYCLE,NULL;
    SET      mine.IF_NEAR,NULL;
}

ACTION wurf_flug {
    RULE      MY.VSPEED += -0.15*TIME_CORR;    // Gravitation
    IF (MY.HEIGHT>0) { END; }
    PLAY_SOUND klonk,0.5,MY;                  // Aufprall auf dem Boden
    SET      MY.HEIGHT,0;
    SET      MY.VSPEED,0;
    SET      MY.EACH_TICK,wurf_rutsch;
}

ACTION wurf_rutsch {
    RULE      MY.SPEED += -0.3*TIME_CORR;
    IF (MY.SPEED>0) { END; }
    SET      MY.TEXTURE,mine_tick_tex;
    SET      MY.EACH_TICK,NULL;
    SET      MY.EACH_CYCLE,mine_scharf;
```



```
SET      MY.TARGET, NULL;  
SET      MY.IF_NEAR, nimm_mine;  
}
```

Um die Mine werfen zu können, müssen wir sie zuerst mit **DROP** auf die Playerposition setzen, denn das unsichtbare Minen-Objekt befindet sich vorher noch an der Stelle, an der wir die Mine aufgenommen haben. Der Minenwurf selbst ist aufgeteilt in eine Flug- und eine Rutschphase. Die Flugbewegung ähnelt der unseres Fasses; allerdings können wir hier, da die Mine ein ACTOR ist, statt eines Objekt-Skills direkt den Parameter **VSPEED** einsetzen. Zusätzlich kommt noch eine Horizontalbewegung (**TARGET MOVE**) hinzu. Würden wir die Flugbahn von der Seite betrachten, so ergäbe sich die klassische Parabel, ein Kennzeichen aller Wurfbahnen in Gravitationsfeldern. **CAREFULLY** muß gesetzt sein, damit die Mine auf ihrem Flug von Wänden und Gegenständen abprallen kann.

Nach dem Aufschlag auf dem Boden rutscht die Mine noch ein Stück weiter und bleibt dann liegen. Die Mine ist jetzt scharf: Sie tickt und lauert darauf, daß ein Gegner - ein Actor mit gesetztem FRAGILE-Flag - in ihre Nähe kommt. Allerdings kann der Player sie auch jederzeit wieder aufnehmen und woanders im Level plazieren.

```
ACTION mine_scharf {  
    SET      SHOOT_FAC, 0;  
    SET      SHOOT_RANGE, 15;  
    EXPLODE   MY;          // Test-Explosion: Gegner innerhalb von 15 Steps?  
    IF (HIT_MINDIST>0)      // was getroffen?  
    { CALL    mine_explode; }  
}
```

```
ACTION mine_explode {  
    SET      MY.PASSABLE, 1;  
    SET      MY.CAREFULLY, 0;      // wichtig für THERE!!  
    SET      MY.SPEED, 0.3;  
    SET      MY.TARGET, FOLLOW;  
    SET      SHOOT_RANGE, 40;  
    SET      SHOOT_FAC, 1;  
    EXPLODE   MY;          // jetzt 'echte' Explosion  
    SET      MY.TEXTURE, min_bums_tex;
```

```
RULE      MY.HEIGHT -= 2;      // Neue Textur ist größer
RULE      THERE.AMBIENT += 0.2; // Region wird durch Blitz erhellt
SET       MY.EACH_CYCLE,mine_weg;
SET       MY.IF_HIT,NULL;
SET       MY.IF_NEAR,NULL;
SET       MY.EACH_TICK,NULL;
IF (RESULT>0)           // wurde Player selbst getroffen?
{ RULE      health -= 200*RESULT; // health abziehen
}

ACTION mine_weg {
  RULE      THERE.AMBIENT -= 0.2; // Region wieder dunkel
  SET       MY.INVISIBLE,1;      // Mine weg
}
```

Bei jedem Animationszyklus prüft die Mine mittels EXPLODE, ob sich ein Opfer in der Nähe befindet. Sie setzt dazu den Skill SHOOT_FAC auf 0, damit niemand verletzt wird. Dies ist ein weiteres Beispiel für den 'Mißbrauch' der EXPLODE-Anweisung zu Testzwecken. Da diese Anweisung relativ viel Rechenzeit kostet, empfiehlt es sich, solche zyklischen Test-Explosionen in größeren Zeitabständen ablaufen zu lassen. Hierzu bietet sich die EACH_CYCLE-Anweisung an.

Eine reale Explosion erleben wir, wenn bei der Testexplosion irgend etwas getroffen wird. Hier ist der SHOOT_RANGE mit 40 Steps deutlich größer gewählt, damit außer dem auslösenden Actor eventuell noch eine Reihe seiner Kampfgenossen mit ins Verderben gerissen werden. Die getroffenen Actors müssen für ihre Beschädigung mit IF_HIT-Aktionen selbst sorgen. Auch den Player kann es erwischen, wenn er sich unvorsichtigerweise in der Nähe der scharfen Mine aufgehalten hat. Dies wird mit dem RESULT-Skill ermittelt.

Den Trick mit FOLLOW, um die Explosionswolke sich ausdehnen zu lassen, kennen wir bereits von unserem simple_tron. Nach erfolgter Explosion verschwindet die Mine. Eine kleine Falle birgt noch das Verändern des Regions-AMBIENT, um den Lichtblitz der Explosion wiederzugeben. Wenn wir auf diese Weise mit dem Schlüsselwort THERE auf eine Region zugreifen, so sollten wir unbedingt dafür sorgen, daß die gleiche Region, die erhellt wurde, auch wieder abgedunkelt wird! Deshalb *muß* hier vorher das Flag CAREFULLY des Actors

auf 0 gesetzt werden, damit sich THERE nicht ändert, wenn die Explosionswolke eine Regionsgrenze überquert.

Freitag: Panels, Menüs, Overlays

Bisher hatten wir uns nur mit der Konstruktion einer virtuellen Welt und ihrer Bewohner befaßt. In diesem Kapitel wollen wir nun solche Funktionen besprechen, die außerhalb dieser Welt liegen, aber dennoch zum Spiel gehören. Dies sind vor allem die Funktionen zur Kontrolle des Spiels und zur Kommunikation des *Users*, der bequem vor dem Bildschirm sitzt, mit seinem imaginären Gegenstück, dem *Player*, der zumeist durch unterirdische Labyrinthes hetzen und die Drecksarbeit erledigen muß.

Texte

Wenn der Player dem User etwas zu sagen hat, kann dies im Klartext erfolgen. Texte dienen der Ausgabe von Meldungen auf dem Bildschirm, können aber auch zum Dialog zwischen Player und Actors in Adventure-Spielen eingesetzt werden. Ähnlich wie bei Bitmaps, Texturen und Objekten gibt es auch bei Texten eine 'Rangfolge' vom rohen Ur-Text bis hin zur aufgepeppten Meldung auf dem Bildschirm.

Der rohe Text wird als Buchstabenfolge mit dem Schlüsselwort **STRING** definiert:

```
STRING    my_string, "Hallo, Welt!!";
```

Der String steht zwischen Anführungszeichen. Er kann auch aus mehreren Zeilen bestehen, die entweder einfach mehreren Zeilen in der WDL-Datei entsprechen oder die durch die Zeichenfolge "\n" voneinander getrennt sind.

```
STRING    my_string_3,  
"Dies ist die erste Zeile,  
dies die zweite,\nund dies endlich die dritte.";
```

Das "\n" zwischen der zweiten und dritten Zeile würde natürlich auf dem Bildschirm nicht erscheinen. Um den Text überhaupt auf dem Bildschirm sichtbar zu machen, müssen wir zunächst einen *Zeichensatz* (Font) dafür definieren. Ein Zeichensatz besteht einfach aus einem Bildchen für jeden Buchstaben. Die

Bildchen sind in einer bestimmten Reihenfolge in einer Bitmap angeordnet. Diese Reihenfolge entspricht der der ersten 128 Zeichen des PC-Zeichensatzes (ASCII-Anordnung):



Abbildung 6 Zeichensatz-Bitmap

Insgesamt muß die Bitmap für 128 Buchstaben, Zahlen und Symbole Platz bieten, wobei alle Zeichen exakt den gleichen Platz einnehmen. Die erste Reihe kann leer bleiben, in der zweiten stehen Zahlen und Sonderzeichen, in der dritten die Groß- und in der vierten die Kleinbuchstaben. Der Zwischenraum zwischen den Zeichen kann mit der transparenten Farbe 0 ausgefüllt werden. Natürlich lassen sich auch beliebige andere Zeichensätze - z.B. griechische - oder Phantasiesymbole darstellen. In unserem Beispiel nehmen die deutschen Umlaute die Stelle der PC-Sonderzeichen '[', '|', ']' und '{', '\', '}' ein, welche auf der PC-Tastatur über die Taste [Alt Gr] zu erreichen sind.

Mit dem Schlüsselwort **FONT** wird der Zeichensatz als solcher definiert:

```
FONT    standard_font,<panfont.pcx>,8,10;
```

Die beiden letzten Parameter geben in Pixeln die Breite und Höhe eines einzelnen Zeichens innerhalb der Zeichensatz-Bitmap an (inklusive den Zwischenräumen). Die Bitmap muß also genau $8 \cdot 10 \cdot 128 = 10240$ Pixel enthalten. Haben wir erstmal einen Font, so können wir aus unserem **STRING** einen **TEXT** machen:

```
TEXT my_text {  
    POS_X      20;  
    POS_Y      40;  
    FONT       standard_font;  
    STRING     my_string;  
}
```

POS_X und **POS_Y** geben in Pixeln die Position des Textes am Bildschirm an, bezogen auf die obere linke Ecke. Dazu müssen wir ihn aber erstmal sichtbar machen, denn der oben definierte Text zeigt sich noch nicht auf dem Bildschirm. Folgende Aktion läßt unseren Text am Bildschirm erscheinen:

```
ACTION show_my_text { SET my_text.VISIBLE,1; }
```

Auf diese Weise könnten wir beliebig viele Texte auf den Schirm zeichnen. Da wir die Text-Position **POS_X** oder **POS_Y** auch während des Spiels zyklisch verändern können, ließe sich unser Text nun auch horizontal oder vertikal über den Bildschirm rollen:

```
ACTION roll_my_text {  
    SET    my_text.POS_Y,400;      // Text unter den Bildschirm setzen  
    SET    my_text.VISIBLE,1;      // sichtbar machen  
    WHILE (my_text.POS_Y>0) {      // Solange Text noch am Bildschirm...  
        WAIT 1;                    // 1/16 Sekunde schlafen  
        RULE my_text.POS_Y -= 1;   // eine Pixelzeile hochrollen  
    }  
    SET    my_text.VISIBLE,0;      // Text abschalten  
}
```

```
IF_R    roll_my_text;
```

Die Aktion startet beim Druck auf die **[R]**-Taste. Wir haben eine **WHILE**-Schleife eingebaut: Die **WAIT**- und **RULE**-Anweisungen werden jeden Tick wiederholt. Erst wenn der **POS_Y**-Wert durch ständiges Addieren von **-1** den Wert **0** erreicht hat, d.h. der Text den oberen Bildrand berührt, wird die **WHILE**-Schleife nicht mehr ausgeführt. Die Aktion endet dann.

Panels

In vielen Games informieren Zahlen- oder Balkenanzeigen über den Zustand des Players oder über bestimmte Spiel-Variablen. Das klassische Beispiel dafür ist das Rollenspiel, in dem Gesundheit, Kampfkraft und Dutzende anderer Player-Eigenschaften sich ständig verändern und Auswirkungen auf das Spielgeschehen haben.

Solche Anzeigen werden in einem **PANEL** auf dem Bildschirm dargestellt. Als Beispiel für ein solches Panel wollen wir jetzt die Inhalte bestimmter Skills als Zahlen auf dem Bildschirm darstellen. Es ist nämlich eine wertvolle Hilfe beim Austesten - *Debugging* - von selbstgeschriebenen Aktionen, wenn wir die Skills, die beim Ablauf der betreffenden Aktion eine Rolle spielen, ständig vor Augen haben.

Panels werden ähnlich definiert wie Texte. Anstelle des String gibt es hier jedoch Definitionen für Anzeige-Elemente wie **DIGITS**, **HBAR** oder **VBAR**:

```
FONT standard_font, <panfont.pcx>, 8, 10;
```

```
PANEL debug_panel {
    POS_X      0;
    POS_Y      12;
    FLAGS      VISIBLE, REFRESH;
    DIGITS      18, 2, 3, standard_font, 160, TIME_FAC;    // Frames pro Sekunde
    DIGITS      50, 2, 2, standard_font, 1, ACTIVE_NEXUS;
    DIGITS      80, 2, 2, standard_font, 1, ERROR;

    DIGITS      100, 2, 1, standard_font, 1, CHANNEL_0;
    DIGITS      110, 2, 1, standard_font, 1, CHANNEL_1;
    DIGITS      120, 2, 1, standard_font, 1, CHANNEL_2;
    DIGITS      130, 2, 1, standard_font, 1, CHANNEL_3;
    DIGITS      140, 2, 1, standard_font, 1, CHANNEL_4;
    DIGITS      150, 2, 1, standard_font, 1, CHANNEL_5;
    DIGITS      160, 2, 1, standard_font, 1, CHANNEL_6;
    DIGITS      170, 2, 1, standard_font, 1, CHANNEL_7;

    DIGITS      200, 2, 4, standard_font, 10, PLAYER_ANGLE;
    DIGITS      240, 2, 4, standard_font, 1, PLAYER_X;
```

```
DIGITS    280,2,4,standard_font,1,PLAYER_Y;
}

STRING debug_labels,
"  fps nex obj err  siz ang  x    y";

TEXT debug_text {
    POS_X    0;
    POS_Y    2;
    FONT      standard_font;
    STRING    debug_labels;
    FLAGS     VISIBLE;
}
```

Nun haben wir am oberen Bildschirmrand eine Menge von Zahlenanzeigen, die uns über den Zustand des Players Auskunft geben. Jede **DIGITS**-Zeile innerhalb der **PANEL**-Definition zeigt einen Skill als Zahlenwert an. Sie enthält 7 Parameter, die die X- und Y-Position der Anzeige, die Anzahl der Stellen, den Zeichensatz, einen Multiplikationsfaktor und den Skill selbst festlegen. Die Positionswerte beziehen sich nicht auf den Bildschirm, sondern sind die Abstände der oberen linken Ecke der Anzeige von der oberen linken Ecke des Panels in Pixeln. Das Flag **REFRESH** bewirkt, daß das Panel ständig neu aufgebaut wird. Nur so können wir es über dem 3D-Fenster darstellen, das ja ebenfalls nach jedem Frame von neuem generiert wird. Bei Texten müssen wir das **REFRESH**-Flag nicht angeben; es ist automatisch gesetzt.

In dem Panel sehen wir nun von links nach rechts die aktuelle Frame-Rate in Bildern pro Sekunde (multipliziert mit 10, um Nachkommastellen sichtbar zu machen), die Auslastung des Nexus (wenn hier der Wert 14 erreicht wird, müssen wir diesen mit dem Schlüsselwort **NEXUS** vergrößern), eine Fehleranzeige (sollte immer 0 anzeigen), die Auslastung der 8 Sound-Kanäle und in 3 weiteren Skills die Player-Position. Wenn wir während des Spiels den Zustand anderer Skills verfolgen wollen, können wir diese hier stattdessen angeben.

Eine kleine Unschönheit hat die Anzeige der Frame-Rate, für die wir den Skill **TIME_FAC** verwendet haben. Dieser Skill hat den Wert 1, wenn die Frame-Rate genau 16 Bilder/Sec. beträgt, und ändert sich proportional. Dadurch wechselt die

Anzeige rasch und läßt sich schlecht ablesen. Mit einem kleinen Trick machen wir die Anzeige träger:

```
SKILL fps {}

ACTION show_panels {
    WHILE (1) { // once CALled, repeat forever
        WAIT 1;
        RULE fps = 0.8*fps + 0.2*TIME_FAC;
    }
}

PANEL debug_panel {
    POS_X 0;
    POS_Y 12;
    FLAGS VISIBLE, REFRESH;
    DIGITS 18, 2, 3, standard_font, 160, fps; // Frames pro Sekunde
    ...
}
```

Unsere 'ewig' wiederholte Aktion **show_panels** ist für die 'Bearbeitung' der Anzeigen-Skills zuständig. Die **RULE**-Anweisung sorgt dafür, daß eine spontane Änderung von **TIME_FAC** sich nur noch zu 20% auf die Anzeige auswirkt, wodurch sich der Wert jetzt fünfmal langsamer ändert.

Ein weiteres Beispiel für Anzeigen sind waagerechte oder senkrechte Balken, die sich gemäß der Werte der dort repräsentierten Skills verschieben. Wir wollen die horizontale Balkenanzeige ausnutzen, um einen Kompaß auf dem Bildschirm darzustellen. Hierzu zeichnen wir uns erst einmal eine Bitmap für den Kompaß-Balken:

```
BMAP kompass_map, <kompass.pcx>, 0, 0, 160, 22;

PANEL kompass_pan {
    POS_X 140;
    POS_Y 376;
    FLAGS VISIBLE, REFRESH;
    HBAR 0, 0, 40, kompass_map, -19.05, PLAYER_ANGLE;
}
```

Der Kompaß zeigt uns immer die Blickrichtung des Players an, wobei 0° Osten entspricht. Die **HBAR**-Zeile ist ähnlich wie **DIGITS** aufgebaut; hier wird wieder zuerst die Position angegeben, dann die Breite der Anzeige in Pixeln, darauf die Bitmap, die verschoben werden soll, schließlich der Multiplikationsfaktor, welcher multipliziert mit dem am Schluß angegebenen Skill die horizontale Verschiebung in Pixeln ergibt.



Abbildung 7 Kompaß-Bitmap

Die Größe unserer Kompaß-Bitmap beträgt 160 Pixel horizontal; die Skala wiederholt sich jedoch bereits nach 120 Pixeln. Bei zyklischen Balkenanzeigen muß die Balken-Bitmap genau um die Anzeigenbreite - hier 40 Pixel - größer sein als ihre maximale Verschiebung. Diese Maximalverschiebung ergibt sich aus dem größtmöglichen Wert des angezeigten Skills, multipliziert mit dem angegebenen Faktor. Auf den Wert **-19.05** kommen wir durch die Überlegung, daß der Kompaß-Balken so gezeichnet ist, daß er bei einer vollen Umdrehung des Players genau um 120 Pixel verschoben werden muß - und zwar nach links, also negativ. Bei einer vollen Drehung hat der Skill **PLAYER_ANGLE** den Wert **2θ** (d.h. 6.3); der gesuchte Faktor ergibt sich daher aus

$$-120 = \text{Faktor} \cdot 6.3 \quad \Rightarrow \quad \text{Faktor} = -120 / 6.3 = \underline{\underline{-19.05}}$$

Overlays - Waffen und Werkzeuge

Die dritte Art von Bildschirm-Objekten, die unsere Engine darstellen kann, ist das Overlay. Ein Overlay kann man sich als Grafik auf einer transparenten Folie vorstellen, die über den Bildschirm gelegt wird. Es eignet sich hervorragend zum Darstellen von Cockpits, Schwertern, Fadenkreuzen etc. über dem 3D-Fenster.

Als Beispiel für eine Waffe, die mit Hilfe eines Overlay realisiert wird, nehmen wir eine Laserkanone, den 'Debugger' aus dem Spiel SKAPHANDER:

```

////////////////////////////////////
// Debugger
////////////////////////////////////
OVLY  cannon1_0 <canon1.LBM>,0,0,68,82;
OVLY  cannon1_1 <canon1.LBM>,68,0,68,80;      // Verschiebung -2
OVLY  cannon1_2 <canon1.LBM>,136,0,68,79;     // -3
OVLY  cannon1_3 <canon1.LBM>,204,0,68,80;     // -2
OVLY  cannon1_5 <canon1.LBM>,68,82,68,82;

OVERLAY  cannon0 { POS_X 27; POS_Y 250; FLAGS ABSPOS; OVLYS cannon1_0; }
OVERLAY  cannon1 { POS_X 25; POS_Y 252; FLAGS ABSPOS; OVLYS cannon1_1; }
OVERLAY  cannon2 { POS_X 24; POS_Y 253; FLAGS ABSPOS; OVLYS cannon1_2; }
OVERLAY  cannon3 { POS_X 25; POS_Y 252; FLAGS ABSPOS; OVLYS cannon1_3; }
OVERLAY  cannon5 { POS_X 27; POS_Y 250; FLAGS ABSPOS; OVLYS cannon1_5; }

```

Eine ganze Menge Overlays für eine einzige Waffe! Aber wir wollen den Debugger animiert mit Rückstoß und Feuerblitz darstellen. Das Schlüsselwort **OVLY** hat genau die gleiche Syntax wie **BMAP**. Jedoch werden Grafiken mit diesem Schlüsselwort auf eine besondere Weise abgespeichert: Sie verbrauchen ungefähr den doppelten bis dreifachen Speicherplatz 'konventioneller' Bitmaps, werden aber dafür wesentlich schneller gezeichnet. Damit können wir auch große Overlays über dem Bildschirm darstellen, ohne daß die Frame-Rate merklich in die Knie geht.

Das Schlüsselwort **OVERLAY** verhält sich zu **OVLY** etwa so wie **TEXTURE** zu **BMAP**. Ein fertiges **OVERLAY** enthält ein oder (in einer späteren Version) auch mehrere **OVLYS**, die in einer ähnlichen Liste wie **BMAPS** in der **OVERLAY**-Definition aufgeführt sind. Hinzu kommen wieder die Positionen **POS_X** und **POS_Y**, die wie bei **TEXT** und **PANEL** den Abstand der oberen linken Ecke des Overlay zur oberen linken Ecke des... nein, eben nicht des Bildschirms, sondern des 3D-Fensters angeben (das Fenster kann kleiner sein als der Bildschirm!). Dies ist ein weiterer Unterschied zwischen **OVERLAY** und **PANEL**. Nur wenn das Flag **ABSPOS** gesetzt ist, beziehen sich die Abstände tatsächlich auf den Bildschirm und nicht auf das 3D-Fenster.



Abbildung 8 Debugger in Aktion

W
i
r
h
a
b
e
n
a
l
s
o
j
e
t
z
t
5
v

erschiedene Waffen-Overlays (wir benutzen nicht alle aus der obigen Bitmap), die wir mit folgender Aktion in Bewegung bringen:

```
SOUND twaeng,<laser.wav>;
```

```
ACTION fire {
    WHILE ((KEY_CTRL>0) || (MOUSE_LEFT>0)) {
        PLAY_SOUND    twaeng,0.3,-0.7;
        SET            cannon0.VISIBLE,0;
        SET            cannon1.VISIBLE,1;
        RULE           PLAYER_LIGHT += 0.5;           // Lichtblitz
        WAIT           1;
        SET            cannon1.VISIBLE,0;
        SET            cannon2.VISIBLE,1;
        RULE           PLAYER_LIGHT -= 0.5;
        WAIT           1;
        SET            cannon2.VISIBLE,0;
        SET            cannon3.VISIBLE,1;
        WAIT           1;
        SET            cannon3.VISIBLE,0;
        SET            cannon5.VISIBLE,1;
        WAIT           1;
        SET            cannon5.VISIBLE,0;
        SET            cannon0.VISIBLE,1;
        WAIT           1;
    }
}

ACTION init_fire {
    SET            cannon0.VISIBLE,1;
    SET            IF_CTRL,fire;
    SET            IF_LEFT,fire;
}
```

Nach Ausführung von **init_fire** kann man die Kanone mit der **[Strg]**-Taste oder der linken Maustaste aktivieren. Man sieht jetzt, warum so viele Overlays gebraucht wurden: Durch zyklisches Wechseln des dargestellten Overlay wird die Kanone animiert. Die Positionen der Overlays unterscheiden sich zusätzlich um ein paar Pixel, um einen Rückstoß anzudeuten. Die **PLAY_SOUND**-Anweisung wird mit dem **BALANCE**-Parameter aufgerufen, da der Kanonenlärm von links kommt.

Wir wären jetzt mit der Overlay-Besprechung theoretisch fertig. Da aber in 3D-Games Schusswaffen erfahrungsgemäß immer wieder eine wichtige Rolle spielen werden, wollen wir unsere Kanone noch soweit vervollständigen, daß sie nicht nur schießt, sondern auch trifft. Dazu brauchen wir noch ein paar Animationen für Trefferblitze und aufsteigende Rauchwölkchen:

```
BMAP    flash_1,<knallig.lbm>,0,0,16,16;
BMAP    flash_2,<knallig.lbm>,16,0,16,16;
BMAP    flash_3,<knallig.lbm>,32,0,16,16;
BMAP    flash_4,<knallig.lbm>,48,0,16,16;

BMAP    wolke_1,<knallig.lbm>,124,0,20,20;
BMAP    wolke_2,<knallig.lbm>,144,0,20,20;
BMAP    wolke_3,<knallig.lbm>,164,0,20,20;
BMAP    wolke_4,<knallig.lbm>,184,0,20,20;
BMAP    wolke_5,<knallig.lbm>,204,0,20,20;

TEXTURE  flash_tex {
    SCALE_XY  12,12;
    CYCLES     4;
    BMAPS      flash_1,flash_2,flash_3,flash_4;
    DELAY      1,1,1,1;
    FLAGS      ONESHOT;
    AMBIENT    1;      // Blitz leuchtet auch in dunklen Regionen!
}

TEXTURE  wolke_tex {
    SCALE_XY  12,12;
    CYCLES     5;
    BMAPS      wolke_1,wolke_2,wolke_3,wolke_4,wolke_5;
    DELAY      2,2,2,2,2;
    FLAGS      ONESHOT;
    AMBIENT    0.5;
}

ACTOR    flash1 {
    TEXTURE    flash_tex;
    FLAGS      GROUND,INVISIBLE,PASSABLE;
}
```

```
ACTOR flash2 {
    TEXTURE    flash_tex;
    FLAGS      GROUND,INVISIBLE,PASSABLE;
}

ACTOR flash3 {
    TEXTURE    flash_tex;
    FLAGS      GROUND,INVISIBLE,PASSABLE;
}

ACTOR flash4 {
    TEXTURE    flash_tex;
    FLAGS      GROUND,INVISIBLE,PASSABLE;
}

ACTION init_flash {
    SET    MY.INVISIBLE,0; // Jetzt sichtbar
    SET    MY.TEXTURE,flash_tex;
    SET    MY.PLAY,1;      // Animation starten
    RULE    MY.DIST = HIT_DIST-0.5; // Trefferdistanz
    RULE    MY.HEIGHT =      // Ungefähre Treffer-Höhe berechnen
        PLAYER_Z - 0.83*HIT_DIST*PLAYER_TILT + RANDOM(0.4) - 1.2;
    DROP    MY;            // Flash vor getroffenen Objekt platzieren
    SET    MY.SPEED,0.3;    // Blitz kommt auf Player zu
    SET    MY.TARGET,FOLLOW;
    SET    MY.EACH_TICK,NULL;
    SET    MY.EACH_CYCLE,init_wolke; // Nach Animation steigt Wolke auf
}

ACTION init_wolke {
    SET    MY.TEXTURE,wolke_tex;
    SET    MY.PLAY,1;
    SET    MY.EACH_TICK,wolke_auf;
    SET    MY.EACH_CYCLE,flash_weg;
}

ACTION wolke_auf {          // Wolke aufsteigen lassen
    RULE    MY.HEIGHT += 0.4*TIME_CORR;
}

ACTION flash_weg {          // Wolke verschwindet
    SET    MY.INVISIBLE,1;
}
```

```

ACTION fire {
  WHILE ((KEY_CTRL>0) || (MOUSE_LEFT>0)) {
    PLAY_SOUND    twaeng,0.3,-0.7;
    SET           cannon0.VISIBLE,0;
    SET           cannon1.VISIBLE,1;
    RULE          PLAYER_LIGHT += 0.5;
    WAIT          1;
    SET           cannon1.VISIBLE,0;
    SET           cannon2.VISIBLE,1;
    SET           SHOOT_RANGE,200;
    SHOOT;                // Schussreichweite 200 steps
    IF (HIT_DIST>0) {      // was getroffen
      SET MY,flash1;       // Synonym direkt setzen
      IF (MY.INVISIBLE)    // ist flash1 unbenutzt?
      { GOTO set_flash; }  // dann jetzt benutzen
      SET MY,flash2;
      IF (MY.INVISIBLE)
      { GOTO set_flash; }
      SET MY,flash3;
      IF (MY.INVISIBLE)
      { GOTO set_flash; }
      SET MY,flash4;
    }
    set_flash:
      CALL    init_flash;
  }
  RULE    PLAYER_LIGHT -= 0.5;
  WAIT    1;
  SET     cannon2.VISIBLE,0;
  SET     cannon3.VISIBLE,1;
  WAIT    1;
  SET     cannon3.VISIBLE,0;
  SET     cannon5.VISIBLE,1;
  WAIT    1;
  SET     cannon5.VISIBLE,0;
  SET     cannon0.VISIBLE,1;
  WAIT    1;
}
}

```

Mit unseren Kenntnissen von der Funktion linearer Automaten dürfte es nicht schwer fallen, diese Aktion zu verstehen. Wir definieren die Trefferblitze der Kanone als Actors. Für jeden Actor stehen zwei **ONESHOT**-Texturanimationen

zur Verfügung. Das Zustandsdiagramm eines solchen kurzlebigen Actors ist sehr simpel (s. Abb.).

Der Actor wird aktiviert, wenn die **SHOOT**-Anweisung in unserer neuen **fire**-Aktion ein Hindernis trifft, d.h. der Skill **HIT_DIST** ungleich 0 ist. Die Aktion **init_flash** setzt die Textur seines ersten Zustandes und platziert ihn an die getroffene Stelle - nämlich 0.5 Steps vor der getroffenen Wand bzw. dem getroffenen Objekt, in Blickrichtung des Players und in Höhe des Zentrums des 3D-Fensters. Die Platzierung erfolgt mit dem **DROP**-Befehl.

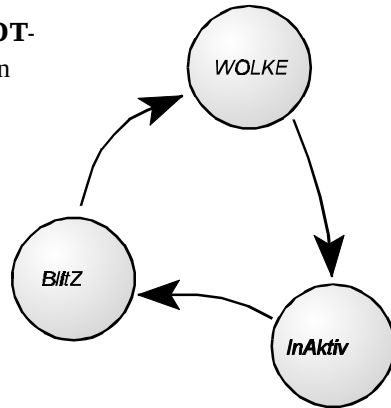


Abbildung 9 Das kurze Leben eines **flash**-Actors

Die **EACH_CYCLE**-Aktion **init_wolke** wird nach Ablauf der Blitz-Animation gestartet. Der Actor verwandelt sich in ein Rauchwölkchen, das mit einer Geschwindigkeit von 0.4 Steps pro Tick himmelwärts steigt. Nach Ablauf der Wolken-Animation wird der Actor **INVISIBLE** gesetzt und ist damit für den nächsten Treffer bereit.

Wir benötigen mehrere **flash**-Actors, denn unsere Waffe feuert kontinuierlich. Während die erste Rauchwolke noch aufsteigt, müssen nebenan schon neue Treffer aufblitzen. In unserer **fire**-Aktion wird deshalb zuerst nachgeschaut, welcher **flash** gerade 'frei' ist, und dieser dann für den nächsten Trefferblitz initialisiert.

Zum Abschluß wollen wir uns noch einer speziellen Art von Overlay widmen, nämlich der Level-Karte. Diese Karte kann per Automapping ständig auf dem 3D-Fenster eingeblendet werden. Damit die Karte aber überhaupt sichtbar wird, müssen wir erst den Skill **MAP_MODE** setzen. Wir tun dies durch eine Ergänzung unserer obigen Panel-Aktion:

```
IF_TAB    NULL;

ACTION show_panels {
    RULE    fps = 0.8*fps + 0.2*TIME_FAC;
    SET     MAP_MODE,0;
    IF (KEY_TAB) {    // [TAB] gedrueckt?
        SET  MAP_MODE,0.5;    // automapping
        IF (KEY_MINUS || KEY_SZ)    // change map scale in steps of 5%
        { RULE  MAP_SCALE = MAP_SCALE*0.95; }
        IF (KEY_PLUS || KEY_APO)    // APO is [+] on US keyboards
        { RULE  MAP_SCALE = MAP_SCALE*1.05; }
    }
}
```

Die Karte wird jetzt bei Druck auf die [TAB]-Taste auf den Bildschirm projiziert, wobei mit den [+] und [-]-Tasten der Maßstab verändert werden kann. Damit das funktioniert, müssen wir zuerst per NULL-Zuweisung die Default-Aktion von der TAB-Taste wegnehmen. **MAP_MODE** 1 würde den gesamten Level zeigen, 0.5 zeigt nur die gesehenen Bereiche. Im Hinblick auf eine internationale Vermarktung unseres Spiels haben wir dabei gleich die US-Tastaturbelegung mit berücksichtigt...

Levelwechsel

Normalerweise besteht unser Game aus mehreren *Leveln*, durch die sich der Player nacheinander hindurchkämpfen muß. Dies stellt uns vor neue Aufgaben. Wir müssen Übergänge zwischen den Leveln schaffen. Wenn der Player in einem niedrigen Level Eigenschaften oder Waffen gesammelt hat, soll er sie in höhere Level mitnehmen können. Außerdem muß man das Spiel jederzeit unterbrechen und dann im gleichen Level und an der gleichen Stelle weiterspielen können. Hierzu benötigen wir eine Möglichkeit zur vollständigen oder teilweisen Speicherung des Spielstands.

Es gibt drei Möglichkeiten, den Level zu wechseln. Zum einen könnten wir verschiedene getrennte Level innerhalb der gleichen WMP-Datei unterbringen. Zwischen diesen Unter-Leveln liegen **border**-Regionen, so daß der Player nicht auf normalem Wege von einem in einen anderen Level spazieren kann. Außerdem

müssen die einzelnen Level räumlich so weit voneinander entfernt sein, daß jeder außerhalb der **CLIP_DIST** des anderen liegt.

Um den Spieler bequem von einem Ort in der Topographie zu einem anderen 'beamen' zu können, ohne mit Koordinaten herumrechnen zu müssen, definieren wir uns am besten ein Pseudo-Objekt als Beam-Ziel und platzieren es mit WED an die Zielposition im betreffenden Level:

```
THING transporter_1 {           // pro Level ein Transporter (oder mehrere?)
    TEXTURE    pseudo_tex;      // irgendeine Textur
    FLAGS    INVISIBLE;         // damit er uns nicht vor der Nase hängt
}

ACTION beam_1 {
    SET    PLAYER_X, transporter_1.X;
    SET    PLAYER_Y, transporter_1.Y;
    SET    PLAYER_ANGLE, transporter_1.ANGLE;
    SET    HERE, zielregion_1;   // Region des transporter_1
    RULE    PLAYER_Z = HERE.FLOOR_HGT + PLAYER_SIZE;
    SET    PLAYER_HGT, 0;
}
```

Die Aktion **beam_1** sorgt dafür, daß der Player sofort am Transporter-Ziel auftaucht. Man kann sie natürlich auch für das Herumbeamen innerhalb des gleichen Levels verwenden! Zu beachten ist, daß wir beim Beamen auch die Player-Region **HERE** neu zuweisen müssen, sonst würde die Kollisionserkennung nicht mehr funktionieren.

Die Methode mit mehreren Leveln in der gleichen WMP-Datei birgt einige Vor- und Nachteile:

Der Levelwechsel geht blitzschnell ohne Disk-Zugriff

Die Maximalgröße der WMP-Datei (ca. 5000 Objekte bei der professionellen Version) beschränkt die Levelgröße und -Anzahl

Das Starten des Spiels dauert länger, da die Texturen für *sämtliche* Level schon bei Spielstart geladen werden müssen.

Eine weitere Möglichkeit besteht darin, für jeden Level eine eigene WMP-Datei zu erstellen. Das Schlüsselwort **MAPFILE** bestimmt dann nur die Topographie des Startlevels; zu weiteren Leveln kann man mit der Anweisung **MAP** innerhalb einer Aktion springen:

```
ACTION beam_2    { MAP  <level_2.wmp>; }
```

Mit **beam_2** wird der Player direkt zur Player-Startposition innerhalb der Datei LEVEL_2.WMP positioniert (wir setzen den Dateinamen in spitze Klammern, damit er später vom Compiler zum Spiel dazugebunden werden kann). Die Walls, Things und Actors werden aus der neuen WMP-Datei gelesen; die restlichen Parameter - Skills, Panels etc. - bleiben unverändert. Der Nachteil dieser Methode ist ein Disk-Zugriff von einigen Sekunden Dauer, um den neuen Level zu laden und aufzubauen. Das Spiel wird dadurch für kurze Zeit 'eingefroren'. Da die WDL-Datei nicht gewechselt wird, müssen auch hier alle Walls, Regionen und sonstigen Objekte sowohl des alten wie auch des neuen Levels in der WDL definiert sein.

Diesen Nachteil kann man vermeiden, wenn man gleich die komplette WDL neu lädt. Dies ist die dritte, aufwendigste Methode des Levelwechsels, die praktisch dem Neustart eines Spiels entspricht. Hierzu dient die Anweisung **LEVEL** (nur verfügbar in der Profi-Version):

```
ACTION beam_3    { LEVEL  "level_3.wdl", "level_3.wrs"; }
```

Aus der kompilierten World Resource LEVEL_3.WRS wird die Datei LEVEL_3.WDL geladen und als neues Spiel gestartet. Die Dateinamen stehen hier übrigens - im Gegensatz zur MAP-Anweisung - zwischen Anführungszeichen, da sie nicht vom Compiler zum Level dazugebunden werden. Alle Objekte, Positionen, Skills und laufenden Aktionen aus dem alten Level werden 'vergessen'; es geht mit der **IF_START**-Aktion in der neuen WDL-Datei weiter. Allerdings gibt es eine Ausnahme, die die **LEVEL**-Anweisung doch von einem kompletten Neustart unterscheidet: Das Verhalten einiger spezieller Skills, nämlich solcher vom Typ **GLOBAL**.

Globale Skills

Man definiert **GLOBAL**-Skills einfach durch das Schlüsselwort **TYPE GLOBAL;** innerhalb der geschweiften Klammern der Skill-Definition. Die globalen Skills sind für 'universelle' Parameter gedacht, die in allen Levels ihre Gültigkeit behalten sollen, z.B. Sound- und Musik-Einstellungen oder Kampfkraft und Ausrüstung des Players. Sie werden beim Levelwechsel nicht 'vergessen', sondern behalten im neuen Level ihre alten Werte bei, auch wenn sie dort mit anderen Werten (**VAL**) neu definiert wurden. Und das müssen sie sein, denn die neue und die alte Level-WDL-Datei muß exakt die gleichen **GLOBAL**-Skill-Definitionen mit den gleichen Skill-Namen in der gleichen Reihenfolge enthalten!

Die Werte aller **GLOBAL**-Skills lassen sich separat auf der Harddisk abspeichern und wieder laden. Hierzu dienen die Anweisungen **SAVE_INFO** bzw. **LOAD_INFO**. Mit diesen Anweisungen kann man allgemeine Parameter - wie Sound- und Musik-Einstellungen, Schwierigkeitsgrad etc. - unabhängig von irgendwelchen Spielständen auf der Harddisk abspeichern.

```
////////////////////////////////////////
SKILL MOTION_BLUR    { TYPE GLOBAL;   VAL 0.5; }
SKILL SOUND_VOL      { TYPE GLOBAL;   VAL 1;  }
SKILL MUSIC_VOL      { TYPE GLOBAL;   VAL 0.5; }
////////////////////////////////////////
SAVEDIR    "C:\\MYGAMES";

ACTION global_save {
    SAVE_INFO "test",0;
}

ACTION global_load {
    LOAD_INFO "test",0;
}
```

SAVE_INFO speichert eine Datei, die die globalen Skill-Werte enthält, im mit **SAVEDIR** angegebenen Verzeichnis unter dem im ersten Parameter angegebenen, maximal fünfstelligen Namen plus einer maximal dreistelligen Zahl (hier 0). Vervollständigt wird der Dateinamen durch die Extension ".SAV". Mit den obigen

Aktionen werden die drei globalen Skills also in der Datei C:\MYGAMES\TEST0.SAV gespeichert bzw. daraus wieder gelesen.

Sollte das angegebene Verzeichnis (doppelte Schrägstriche \\ beachten!) beim Speichern noch nicht existieren, so wird es neu angelegt. Und falls der Befehl **LOAD_INFO** die angegebene Datei nicht findet, beläßt er einfach die **GLOBAL-Skills** so, wie sie sind.

Es bietet sich an, die Aktion **global_load** bei Spielstart und **global_save** vor Spielende auszuführen. Dann 'erinnert' sich das Spiel an alle Veränderungen der **GLOBAL-Skills**, die wir z.B. durch Verändern eines Lautstärke-Schiebereglers im Panel durchgeführt haben, und behält die einmal eingestellte Lautstärke beim Neustart bei.

Wie erreichen wir übrigens, daß nur *einige*, aber nicht *alle* globalen Skills mit **LOAD_INFO** von der Harddisk gelesen werden? Ganz einfach: Wir kopieren die Werte derjenigen Skills, die nicht verändert werden sollen, in 'Schatten-Skills', um sie nach **LOAD_INFO** wieder zurückzuholen. Wenn wir also in der obigen Lese-Aktion den Skill **MOTION_BLUR** nicht verändern wollen, sieht das so aus:

```
SKILL motion_blur_s { } // Schatten-Skill für MOTION_BLUR

ACTION global_load_vols {      // Soll nur SOUND_VOL und MUSIC_VOL lesen
    SET    motion_blur_s,MOTION_BLUR; // alten Wert zwischenspeichern
    LOAD_INFO "test",0;
    SET    MOTION_BLUR,motion_blur_s; // alten Wert zurückholen
}
```

Noch ein Hinweis: Beim Herumexperimentieren mit dem Speichern und Laden von globalen Skills werden sich erfahrungsgemäß hin und wieder erschreckende, absturz-ähnliche Effekte einstellen. Das passiert immer dann, wenn man nach dem letzten Speichern die Anzahl der globalen Skills verändert hat. Also: nach jedem Editieren der WDL alle gespeicherten alten Skill-Dateien sicherheitshalber löschen!

Spielstände

Mit der Anweisung **SAVE** wird auf einen Schlag der komplette Zustand des Spiels auf Harddisk abgespeichert. Dies betrifft folgende Parameter:

Die Namen der aktuellen .WDL- und .WMP-Datei,
der Wert aller normalen und **GLOBAL**-Skills,
der Inhalt aller Synonyme (**MY**, **THERE** etc.),
den Zustand der Listen **EACH_TICK**, **EACH_SEC**, **LAYERS**, **PANELS**,
MESSAGES,
alle Key-Aktionen (**IF_F1** etc.),
die Positionen und sonstigen veränderbaren Parameter jedes **PANEL**, **TEXT**
oder **OVERLAY**,
die Koordinaten und sonstigen Parameter sämtlicher Actors,
die Flags aller Regionen, Walls und Things,
die Koordinaten und sonstigen Parameter jeder Region, jeder Wall und jedes
Things, dessen Flag **SAVE** gesetzt ist. Dieses Flag muß bei allen Objekten und
Regionen gesetzt sein, die sich während des Spiels in irgendeiner Form
verändern!

Die gespeicherten Spielstände sind verschlüsselt und komprimiert, so daß sie nicht viel Platz auf der Harddisk einnehmen. Mit der Anweisung **LOAD** wird ein gespeicherter Spielstand wieder entschlüsselt und geladen. Alle im Moment des Abspeicherns laufenden **EACH_TICK**-Aktionen werden nach dem Laden des Spielstandes von Beginn an neu gestartet. Befindet sich der Player zum Zeitpunkt des Ladens in einem anderen Level, so wird automatisch wie mit einer **LEVEL**-Anweisung der Level gewechselt. Das Laden selbst wird - ebenso wie die Anweisungen **LEVEL** und **MAP** - nicht unmittelbar in der Aktion, in der die entsprechende Anweisung steht, sondern erst nach Ende der Aktion durchgeführt.

Für die Spielstände gilt übrigens das schon oben für globale Skills gesagte: Laden eines alten Spielstands nach Verändern der WDL-Datei führt... nein, vielleicht nicht unbedingt zum Absturz, aber auf jeden Fall zu einem gezielten Abbruch des Spiels mit entsprechender Fehlermeldung.

Wir wollen nun all unsere bisherigen Erkenntnisse nutzen, um eine Aktion zum Speichern und Laden von Spielständen zu definieren. Mit der Taste [F2] wollen wir den Spielstand abspeichern, mit [F3] ihn wieder zurückladen. Das hört sich noch ziemlich einfach an. Damit das Ganze aber nicht zu langweilig wird, wollen wir durch wiederholtes Drücken von [F3] zwischen den drei letzten gespeicherten Spielständen wählen können - und zwar auch noch *nach* dem Laden eines dieser Spielstände. Eine Meldung am Bildschirm soll die Nummer des jeweiligen Spielstandes angeben.

```

////////////////////////////////////
// Spielstand-Verwaltung
////////////////////////////////////
SKILL slot    { TYPE GLOBAL; VAL 1; } // Nummer des Spielstands (1..3)
////////////////////////////////////
SKILL number  { VAL 0; } // Nummer zum Laden
////////////////////////////////////
SAVEDIR      "C:\\MYGAMES";

STRING leer, " ";
STRING ok, "OK";
STRING save_yesno, "Spielstand sichern (J/N)?";
STRING load_yesno1, "Letzten Spielstand laden (J/N/F3)?";
STRING load_yesno2, "Vorletzten Stand laden (J/N/F3)?";
STRING load_yesno3, "Drittletzten Stand laden (J/N/F3)?";
STRING wait_txt, "Bitte warten...";
STRING save_nix,
"FEHLER - Nix gesichert!\\nHarddisk schon voll?";
STRING load_nix,
"Kein Spielstand gefunden!\\nMerke:\\nErst sichern - dann laden!";

////////////////////////////////////
FONT    standard_font, <panfont.pcx>, 8, 10;

TEXT screen_txt {
    POS_X      30;
    POS_Y      24;
    FONT    standard_font; // s.o.
    STRING    leer;
}
////////////////////////////////////

```



```
// Message-Aktionen
////////////////////////////////////
ACTION show_message {           // Meldung anzeigen
    SET    screen_txt.VISIBLE,1;
    WAITT  80;                  // 5 Sekunden warten
    SET    screen_txt.VISIBLE,0;
}

ACTION wait_yesno {             // Auf J/N-Tastenbetätigung warten
    SET    screen_txt.VISIBLE,1; // Frage anzeigen
    SET    MOVE_MODE,0;          // Player einfrieren
    SET    IF_N, clear_yesno;    // Tasten belegen
    SET    IF_ESC, clear_yesno;
}

ACTION clear_yesno {            // J/N-Abfrage beenden
    SET    screen_txt.VISIBLE,0; // Text weg
    SET    MOVE_MODE,1;          // Player wieder auftauen
    SET    IF_J, NULL;           // Tasten ent-belegen
    SET    IF_Y, NULL;
    SET    IF_Z, NULL;
    SET    IF_N, NULL;
    SET    IF_ESC, NULL;
    SET    IF_F3,qload_game1;    // F3 lädt wieder den letzten Spielstand
}

////////////////////////////////////
// Speichern & Laden
////////////////////////////////////
ACTION qsave_game {             // Spielstand speichern, mit Abfrage
    SET    screen_txt.STRING,save_yesno;
    SET    IF_J, save_game;
    SET    IF_Y, save_game;
    SET    IF_Z, save_game;
    CALL   wait_yesno;
}

ACTION load_slot {              // Liest slot-Skill von Harddisk
    LOAD_INFO "INFO",0;
}

ACTION save_game {              // Spielstand direkt speichern
    CALL   clear_yesno;
```

```
CALL    load_slot;    // globalen Skill slot laden
RULE    slot += 1;    // und um 1 erhöhen
IF (slot>3)    // Wenn >3, dann wieder auf 1
{ RULE    slot -= 3; }
SAVE_INFO "INFO",0;    // neuen slot-skill wieder speichern
SET     RESULT,0;    // Um Fehler zu erkennen
SET     screen_txt.STRING,ok;
CALL    show_message;
SAVE    "GAME",slot; // Game abspeichern
IF (RESULT<0)    // Fehler beim Speichern?
{ SET     screen_txt.STRING,save_nix; }
CALL    show_message;    // show error message
}
```

```
ACTION qload_game1 {
    SET     number,0;    // nimm letzten Spielstand
    SET     screen_txt.STRING,load_yesno1;
    SET     IF_Y,load_game;
    SET     IF_Z,load_game;
    SET     IF_J,load_game;
    SET     IF_F3,qload_game2;
    CALL    wait_yesno;
}
```

```
ACTION qload_game2 {
    SET     number,1;    // vorletzten Spielstand
    SET     screen_txt.STRING,load_yesno2;
    SET     IF_Y,load_game;
    SET     IF_Z,load_game;
    SET     IF_J,load_game;
    SET     IF_F3,qload_game3;
    CALL    wait_yesno;
}
```

```
ACTION qload_game3 {
    SET     number,2;    // drittletzten Spielstand
    SET     screen_txt.STRING,load_yesno3;
    SET     IF_Y,load_game;
    SET     IF_Z,load_game;
    SET     IF_J,load_game;
    SET     IF_F3,qload_game1;
    CALL    wait_yesno;
}
```

```
ACTION load_game {
    CALL  clear_yesno;
    SET   screen_txt.STRING,wait_txt;
    SET   screen_txt.VISIBLE,1;
    WAIT  1;          // damit message noch vor Laden sichtbar wird
    CALL  load_slot;   // globalen Skill slot laden
    RULE  slot -= number;
    IF (slot<1)
    { RULE  slot += 3; }
    LOAD  "GAME",slot;
    SET   screen_txt.STRING,load_nix;    // schiefgegangen!
    CALL  show_message;    // Fehlermeldung
}

////////////////////////////////////
IF_F2  qsave_game;
IF_F3  qload_game1;
```

Für unsere scheinbar einfache Aufgabe brauchen wir also jede Menge Aktionen. Wir werden noch sehen, daß eines der Probleme darin besteht, auch nach dem Laden irgendeines neuen Spielstandes sich an die Nummer des zuletzt gespeicherten Stands zu erinnern. Da das Laden eines Spiels sämtliche Skills verändert - natürlich eben auch die Nummer des zuletzt gespeicherten Spiels - , müssen wir also diese Nummer separat auf Harddisk abspeichern. Die einzige Möglichkeit dazu ist die Verwendung eines globalen Skills, den wir hier **slot** genannt haben.

Zuerst definieren wir uns ein paar Aktionen, um Meldungen auf dem Bildschirm auszugeben oder auf eine Ja/Nein-Tastenbetätigung zu warten. **show_message** zeigt den Text **screen_txt** für 5 Sekunden am Bildschirm. **wait_yesno** bringt den gleichen Text, wartet jetzt aber auf eine Tastenbetätigung als Antwort vom User. In der Zwischenzeit wird durch Setzen des vordefinierten Skills **MOVE_MODE** auf 0 das Spiel 'eingefroren'. Die Aktion, die bei einer 'Ja'-Antwort ausgeführt werden soll, muß zunächst einen **CALL clear_yesno** durchführen, damit der Player wieder 'aufgetaut' und die Tasten entbelegt werden. Betätigen der Tasten [N] oder [Esc] führt **clear_yesno** direkt aus. Die 'Ja'-Aktion legen wir nicht nur auf die [J]-Taste, sondern auch auf [Y] und [Z], um auch US-Tastaturen zu berücksichtigen.

Mit der [F2]-Taste wird die Aktion **qsave_game** ausgeführt. Durch Betätigen der [J]-Taste nach der Sicherheitsabfrage starten wir dann die eigentliche Speicher-Aktion **save_game**. Zuerst wird der globale Skill **slot** aus der Datei INFO0.SAV gelesen. Dieser Skill enthält die Nummer des zuletzt gespeicherten Spielstandes. Er wird um 1 erhöht, bei Überschreiten seinen Maximalwertes 3 gegebenenfalls wieder auf 1 zurückgesetzt und erneut in der Datei INFO0.SAV gespeichert. Anschließend erfolgt das eigentliche Speichern des Spielstandes unter dem Namen GAME0.SAV, GAME1.SAV oder GAME2.SAV - je nachdem, welchen Wert **slot** gerade hat. Bei Auftreten eines Fehlers beim Speichern erhält der vordefiniert Skill **RESULT** automatisch einen Wert ungleich 0. Wir geben in diesem Fall eine Fehlermeldung und ansonsten "OK" mit Hilfe der Aktion **show_message** aus.

Auch beim Laden mit der [F3]-Taste erfolgt zunächst die Frage an den User. Hier kann man aber nicht nur mit [J] und [N], sondern auch durch nochmaliges Drücken der [F3]-Taste antworten. Jedes Betätigen von [F3] bringt eine andere Meldung auf den Bildschirm, ändert den Skill **number** und legt zyklisch eine der drei Aktionen **qload_game1**, **qload_game2** oder **qload_game3** auf [F3].

Betätigen von [J] führt dann die Aktion **load_game** aus. Diese Aktion soll vor dem Laden des Spiels, das bei einem erforderlichen Levelwechsel ziemlich lange dauern kann, die Meldung "Bitte warten..." auf dem Bildschirm ausgeben. Leider aber erfolgt die Darstellung eines **TEXT** erst beim nächsten Bildaufbau. Wir würden also die Meldung niemals sehen, denn die **LOAD**-Anweisung unterbricht den Bildaufbau so lange, bis der Level komplett neu aufgebaut wurde. Deshalb führt **load_game** eine **WAIT**-Anweisung durch. Der **wait_txt** steht nun auf dem Bildschirm.

Um die Nummer des zu ladenden Spielstands zu ermitteln, wird **slot** wieder aus der globalen Skill-Datei gelesen und der Skill **number** davon abgezogen. Der Wert von **number** gibt an, ob der letzte (0), der vorletzte (1) oder der drittletzte (2) Spielstand geladen werden sollen. Jetzt endlich kann das Laden selbst erfolgen.

Zum Abschluß eine kleine Denksportaufgabe: Wir müssen nach **LOAD** nicht erst den Skill **RESULT** auswerten, um festzustellen, ob beim Laden der Datei irgendetwas schiefgegangen ist. Warum wohl?

Index

ABSPOS	71	FORCE_ROT	10
ACCEL	17	FORCE_STRAFE	10
ACCELERATION	24	FORCE_UP	21
ADDT	57	FRAGILE	44, 61
ALIGN HOR	6	FRICTION	17, 20
ASCII	65	GLOBAL	81, 83
Automap	77	GLOBAL-Skills	81
BALANCE	73	HBAR	67, 70
beamen	79	HERE	25
BERKELEY	28	HIT_DIST	52, 77
BIND	2	Hysteresis	32, 40
BULLET	45	IF	20
CALL	87	IF_ARRIVED	45
CAREFULLY	52, 54, 61, 62	IF_ENTER	25
CLIP_DIST	4	IF_FAR	31, 44
DELAY	30	IF_HIT	44, 45
Denksportaufgabe	88	IF_LEAVE	25
DIGITS	67, 68, 70	IF_NEAR	27, 31, 44
DIST	44	IF_START	2, 10
DROP	61, 77	INCLUDE	2, 32
EACH_CYCLE	44, 52, 77	INERTIA	17
EACH_TICK	10, 44, 45	Joystick	10
Erdbeben	26	Karte	77
EXPLODE	44, 54, 56, 62	LEVEL	80
Explosion	62	Levelwechsel	78
FADE_PAL	40	Lifte	33
FLOOR_HGT	19	LOAD	88
Flügeltüren	36	LOAD_GAME	83
FOLLOW	52, 53	LOAD_INFO	81
FONT	65	MAP	80
FORCE_AHEAD	10	MAP_MODE	77

MAPFILE	2, 80	SHOOT_FAC	56
MIRROR	50	SHOOT_RANGE	52
MOVE	61	SHOOT_Y	52
MOVE_MODE	87	SKAPHANDER	45, 55
MY	32, 51	Special Effects	55
Nebel-Effekt	40	Spielstände	83
Nexus	68	STRING	64, 65
ONESHOT	30, 76	TARGET	45, 61
Overlay	59, 71	TEXT	65
OVLY	71	THERE	35, 62
PALETTE	3	TIME_CORR	17, 57
PANEL	67	TIME_FAC	68
PASSABLE	54	TRANSPARENT	32
PLAY_SOUND	73	Tücken	16
PLAYER_COS	12	TYPE	81
PLAYER_HGT	19	VBAR	67
PLAYER_SIN	12	VIDEO	2
PLAYER_SIZE	19, 21	VSPEED	52, 61
PLAYER_TILT	23	WAITT	27
PLAYER_VROT	11	WAY	28, 45
PLAYER_VX	11	WHILE	66
PLAYER_VY	11	Wurf	60
PLAYER_Z	19		
PORTCULLIS	4		
RANDOM	45		
REFRESH	68		
Regelschwingungen	17		
RENDER_MODE	57		
RESULT	56, 57, 88		
SAVE	83		
SAVE_INFO	81		
SAVEDIR	81		
SHOOT	44, 52, 56, 77		